Pontificia Universidad Javeriana
Facultad de Ingeniería — Ingeniería de Sistemas

Software Design Document
SDD

GammaLab v2.0

Proyecto	Plataforma de análisis y visualización de señales neurofisiológicas Gamma Lab
Documento	Software Architecture Document — SDD
Versión	2.0
Estado	En elaboración
Fecha	27 de agosto de 2026

Autores

Juan Felipe Romero Ortiz — `juanromeroo@javeriana.edu.co`
Laura Juliana Garzón Arias — `garzonlj@javeriana.edu.co`
Xiomara Herrera Suárez — `x_herrera@javeriana.edu.co`
Sebastián Almanza Galvis — `s.almanzag@javeriana.edu.co`

Nota de uso. Documento académico/interno del equipo de proyecto. No distribuir sin autorización.
Trazabilidad. Este SDD detalla la implementaci´on t´ecnica, estructura modular y dise˜no funcional de Gamma Lab, en coherencia con el SAD y el SRS previamente definidos.

Índice

1. Introducción **5**

1.1. Propósito del documento 5

1.2. Alcance 5

1.3. Audiencia 5

2. Arquitectura **7**

2.1. Vista Lógica 7

2.1.1. Propósito 7

2.2. Vista Física 9

2.2.1. Propósito 9

2.2.2. Relaciones 10

2.2.3. Versiones de las librerías 10

2.3. Vista de Procesos 10

2.3.1. Propósito 10

2.4. Arquitectura de implementación 12

2.4.1. Organización del código y reglas de dependencia 12

2.4.2. Núcleo y ciclo de vida 14

2.4.3. Arranque y cierre 15

2.4.4. Hilos, tareas y recursos 16

2.4.5. Fallos, registro y mensajes 18

2.4.6. Extensibilidad 19

3. Diseño Detallado de Componentes **20**

3.1. Modelo de datos y almacenamiento 21

3.1.1. TrialDataset 21

3.1.2. SignalDataset 22

3.1.3. DataStore 22

3.1.4. Archivo de proyecto `.glab` 23

3.1.5. Diagrama de clases del modelo de datos 24

3.2. Servicios de soporte 25

3.2.1. SettingsService 25

3.2.2. FileIOService 25

3.2.3. MeasurementService 26

3.2.4. ExportService 27

3.2.5. ProjectService 27
3.2.6. Diagrama de clases de los servicios de soporte 28
3.3. Presentación 29
3.3.1. Integración Qt/VTK 29
3.3.2. Retroalimentación al usuario 30
3.3.3. VTKContextMenu 30
3.3.4. MainWindow 31
3.3.5. Diagrama de clases de la presentación 32

4. Comportamiento del sistema **33**
4.1. Diagrama abrir proyecto 33
4.2. Diagrama guardar proyecto 35
4.3. Diagrama cerrar proyecto 37
4.4. Diagrama wavelet 39

5. Interfaz de usuario **41**
5.1. Organización de la pantalla principal 41
5.2. Carga y guardado de archivos (pestaña *File*) 44
5.3. Preprocesamiento 45
5.4. Analysis 47

Índice de figuras

1. Vista lógica del sistema GammaLab 2.0: núcleo, servicios y familias de plugins. . 7
2. Vista física de GammaLab 2.0: estación única de escritorio y su superficie de entrada/salida. 9
3. Flujo maestro de sesión de GammaLab 2.0, con carriles Investigador/Sistema. . . 11
4. Diagrama de clases del modelo de datos y almacenamiento 24
5. Diagrama de clases de los servicios de soporte 28
6. Diagrama de clases de los elementos de presentación 32
7. Diagrama de secuencia para abrir proyecto 33
8. Diagrama de secuencia para guardar proyecto 35
9. Diagrama de secuencia para cerrar proyecto 37
10. Diagrama de secuencia para wavelet 39
11. Organización de la pantalla principal de Gamma Lab 41
12. opciones de slope 42
13. opciones de eliminar medida 42
14. opciones de zoom 42
15. explorador de archivos desplegado 43
16. resultados desplegados 43
17. Los resultados pueden ser exportados en formato csv. 43
18. Pestaña File de Gamma Lab 44
19. Flujo de carga de la señal (*Open Signal*) 44
20. Flujo de carga de proyecto (*Open Project*) 45
21. Pestaña Preprocessing de Gamma Lab 45
22. opciones de trials 46
23. opciones de remove artifact 46
24. opciones de filter 47
25. opciones de FFT 47
26. opciones de FFT Average 48
27. opciones de PSD 48
28. opciones de PSD Average 49
29. opciones de Average 49
30. opciones de Event related potential 50
31. opciones de Wavelet 50
32. opciones de Wavelet Average 51
33. opciones de PAC 51

Índice de cuadros

1. Descripción de componentes de la Vista Lógica 7
2. Paquetes de primer nivel y componentes que alojan 12
3. Correspondencia ADR / principio → sección del SDD 13
4. Eventos publicados en el canal del Kernel 15
5. Severidades y tratamiento 18
6. Contrato `IPlugin` 19
7. Campos de `TrialDataset` 21
8. Estructuras de trials en `SignalDataset` 22
9. Claves usadas en el `DataStore` 23
10. Campos del manifiesto `.glab` 24
11. Preferencias guardadas por `SettingsService` 25
12. Interpretación de los formatos de archivo 25
13. Tipos de medición 26
14. Exportación de tablas 27
15. Pasos de las operaciones de proyecto 28
16. Utilidades de integración Qt/VTK 29
17. Presentación de los mensajes por severidad 30
18. Contenido del menú contextual de las gráficas 30
19. Responsabilidades de `MainWindow` 31

1. Introducción

1.1. Propósito del documento

El presente **Software Design Document (SDD)** tiene como propósito documentar el diseño técnico de GammaLab v2.0, describiendo la organización de sus componentes, las responsabilidades de sus módulos, las relaciones entre ellos y los mecanismos que permiten su interacción. El documento profundiza en las decisiones de diseño que orientan la implementación del sistema, complementando la descripción arquitectónica establecida en el **Software Architecture Document (SAD)** y los requisitos definidos en el **Software Requirements Specification (SRS)**.

Asimismo, el SDD proporciona una referencia técnica para comprender la estructura interna del software, analizar sus flujos de ejecución y orientar el desarrollo, mantenimiento y validación de sus funcionalidades.

1.2. Alcance

El alcance de este documento comprende la descripción de la arquitectura y el diseño detallado de GammaLab v2.0, desde la organización general de sus componentes hasta los elementos técnicos que intervienen en su funcionamiento. Para ello, incluye los siguientes aspectos:

- **Arquitectura del sistema:** Descripción de las vistas de escenarios, lógica, de despliegue, de procesos y física, así como de los principales componentes y sus responsabilidades.
- **Estructura del sistema:** Documentación de las clases, atributos, métodos y responsabilidades asociadas a los componentes principales.
- **Comportamiento del sistema:** Descripción de los flujos de ejecución y de las interacciones entre los componentes durante las operaciones principales.
- **Persistencia y modelos de datos:** Identificación de las estructuras utilizadas para representar y almacenar las señales, los ensayos *(trials)* y otros datos relevantes del sistema.
- **Relaciones entre componentes:** Descripción de las dependencias, los mecanismos de comunicación y la utilización de los servicios compartidos.
- **Interfaz de usuario:** Descripción de la organización de la interfaz, sus elementos de navegación y las principales funcionalidades disponibles para la gestión, el preprocesamiento y el análisis de señales.

1.3. Audiencia

La audiencia principal de este documento incluye:

- **Desarrolladores de software:** Proporciona información sobre la estructura interna del sistema, las responsabilidades de las clases, las dependencias entre componentes y los criterios de diseño que orientan la implementación y el mantenimiento.
- **Equipo de QA:** Sirve como referencia para contrastar el diseño y el comportamiento del sistema con los requisitos especificados, así como para identificar posibles inconsistencias entre lo definido y lo implementado.

- **Directora de tesis:** Permite evaluar la coherencia técnica del diseño, su correspondencia con los requisitos establecidos y el avance de las funcionalidades contempladas para GammaLab v2.0.
- **Stakeholders:** Ofrece una descripción técnica general de la solución, sus principales componentes y las funcionalidades que contempla, facilitando la comprensión de su organización y evolución.+

2. Arquitectura

Esta sección consolida la arquitectura de GammaLab 2.0 desde la perspectiva del Software Design Document (SDD), alineada con el Software Architecture Document (SAD). Su propósito es ofrecer una visión clara de los componentes, relaciones, flujos y decisiones que soportan el diseño e implementación descritos posteriormente en el Diseño Detallado.

A continuación, se desarrollan las vistas específicas que articulan la arquitectura hacia el diseño e implementación del sistema.

2.1. Vista Lógica

2.1.1. Propósito

La vista lógica describe los componentes más importantes del sistema sin detallar a profundidad su instalación en hardware. Su objetivo es proporcionar una representación clara de cómo se organizan las responsabilidades y cómo interactúan los módulos fundamentales de GammaLab 2.0.

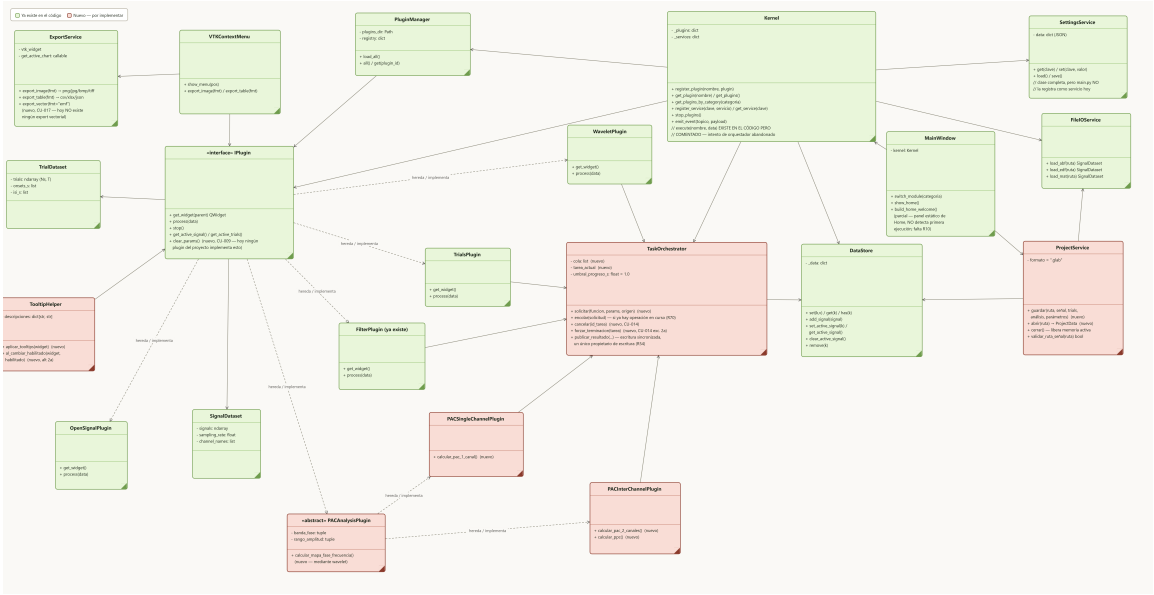


Figura 1: Vista lógica del sistema GammaLab 2.0: núcleo, servicios y familias de plugins.

La Figura 1 ilustra los principales componentes del software y sus interfaces. La Tabla 1 describe cada uno de ellos.

Cuadro 1: Descripción de componentes de la Vista Lógica

Componente	Descripción	Relaciones / Dependencias
Kernel (núcleo)	Orquestador central de la aplicación: administra el registro de plugins y servicios, coordina la ejecución y media la comunicación entre módulos.	Provee `register_service()`/`get_service()` y `register_plugin()`; es consumido por `MainWindow` y por todos los plugins.
PluginManager / `loader`	Descubre e instancia los plugins disponibles a partir de sus carpetas y archivos `properties.yml`.	Utiliza `IPlugin` como contrato; registra cada instancia en el Kernel.

Componente	Descripción	Relaciones / Dependencias
IPlugin (interfaz)	Contrato que toda extensión del sistema implementa (`initialize`, `start`, `stop`, `process`, `get_widget`).	Es implementada por todos los plugins; el Kernel la consume de forma polimórfica.
DataStore	Almacén centralizado en memoria de señales, trials, resultados intermedios y metadatos.	Consumido por el Kernel, los plugins y `FileIOService` mediante claves canónicas.
FileIOService	Lee archivos de señal en formato ABF, EDF y MAT, y los convierte a la estructura interna `SignalDataset`.	Utiliza las librerías `EEGReaders` (`pyabf`, `pyedflib`); publica el resultado en `DataStore`.
ExportService	Genera archivos de salida (imagen y tabla) a partir de las visualizaciones activas.	Utiliza `VTK` para capturar el render; es consumido por los plugins desde el menú contextual.
MeasurementService	Gestiona las mediciones interactivas de pendiente y amplitud sobre los gráficos.	Utiliza `VTK` para la interacción; persiste sus resultados en `DataStore`.
SettingsService	Administra la configuración general del sistema y las preferencias del usuario.	Persiste en `gamma_lab_settings.json`; consumido por el Kernel y los plugins.
ProjectService	Gestiona la persistencia del espacio de trabajo – señal activa, trials y análisis – en archivos de proyecto `.glab`.	Consume `DataStore` para serializar y restaurar el estado completo de la sesión.
TaskOrchestrator	Centraliza la ejecución de cómputo pesado (PAC, Wavelet, PSD) fuera del hilo de interfaz, con soporte de cancelación.	Es consumido por los plugins de análisis; se apoya en multiprocesamiento y multithreading.
Plugins (extensible)	Punto de extensión del sistema, organizados por categoría (E/S, Preprocesamiento, Análisis, Medición).	Consumen los servicios del Kernel; dependen de `PyQt5` y `VTK` para su interfaz.
MainWindow (APP)	Ventana principal de la aplicación; gestiona la navegación entre categorías de plugins y aloja su interfaz.	Consume el Kernel para descubrir y mostrar los plugins registrados.
PyQt5 (framework)	Framework de interfaz gráfica que administra ventanas, widgets y eventos del usuario.	Utilizado por el Kernel, `MainWindow` y los plugins para construir la interfaz.
VTK (librería)	Librería de visualización científica 2D/3D.	Usada directamente por los plugins de análisis y por `ExportService`/`MeasurementService`.
EEGReaders (librería)	Conjunto de librerías (`pyabf`, `pyedflib`) para leer formatos de señal biomédica.	Utilizado por `FileIOService`.
ScientificLibs (librería)	Conjunto de librerías científicas (NumPy, SciPy, PyWavelets) para el cómputo numérico.	Utilizado por los plugins de análisis y por `TaskOrchestrator`.

El *Kernel* centraliza el registro y consumo de servicios mediante `register_service()`/`get_service()`; el *DataStore* soporta la comunicación entre módulos mediante claves canónicas. El código se organiza en paquetes correspondientes a estas mismas responsabilidades (`io`, `preprocessing`, `analysis`, `measure`), lo que mantiene la correspondencia entre el diseño lógico y la implementación.

2.2. Vista Física

2.2.1. Propósito

La vista física representa cómo se despliega y ejecuta GammaLab 2.0. Presenta los componentes físicos más importantes del sistema, sobre los cuales se instalan los componentes de software.

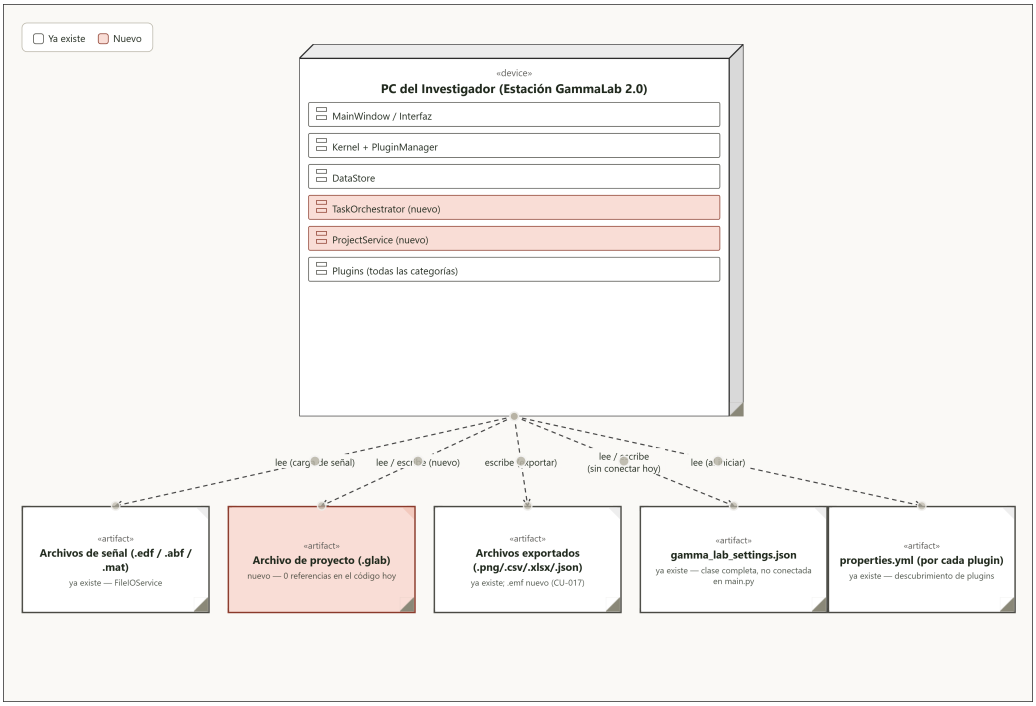


Figura 2: Vista física de GammaLab 2.0: estación única de escritorio y su superficie de entrada/salida.

La Figura 2 presenta la distribución física de los componentes del sistema en el entorno objetivo de ejecución. A continuación se describen los elementos principales:

- **«device» PC**: estación de trabajo del investigador donde se despliega y ejecuta el sistema completo; no existe arquitectura cliente-servidor.
- **«OS» Windows 10/11**: sistema operativo base; versión mínima Windows 10 de 64 bits.
- **«executionEnvironment» Python Runtime 3.11**: entorno de ejecución que sostiene la aplicación y sus librerías; versión mínima 3.11.x.
- **«artifact» GammaLab.exe**: ejecutable principal, generado con PyInstaller, que inicia la aplicación y coordina la carga de todos los componentes.
- **«artifact» Plugins (*.py)**: archivos de plugin cargados dinámicamente por el Kernel durante la ejecución.
- **«artifact» Archivos de señal** (`.edf`/`.abf`/`.mat`): leídos por `FileIOService`.
- **«artifact» Archivo de proyecto** (`.glab`): generado y leído por `ProjectService`.
- **«artifact» gamma_lab_settings.json**: persistencia de configuración gestionada por `SettingsService`.
- **«artifact» properties.yml** (por cada plugin): sostiene el descubrimiento automático de plugins.

2.2.2. Relaciones

- La relación *loads* indica que el ejecutable principal carga y ejecuta los archivos de plugin durante el tiempo de ejecución.
- La relación *uses* especifica que el ejecutable principal utiliza los servicios y librerías desplegados dentro del mismo entorno de ejecución.

2.2.3. Versiones de las librerías

Las versiones ancladas para las librerías utilizadas, tal como se fijan en `requirements.txt`, son las siguientes:

- **PyQt5** = Versión 5.15.11 (`PyQt5-Qt5` 5.15.2)
- **ScientificLibs**
  - NumPy = Versión 2.3.4
  - Pandas = Versión 2.3.3
  - SciPy = Versión 1.16.2
  - PyWavelets = Versión 1.9.0
- **VTK** = Versión 9.5.2
- **EEGReaders**
  - pyABF = Versión 2.3.8
  - pyEDFlib = Versión 0.1.42
- **Otras**
  - h5py = Versión 3.15.1
  - PyYAML = Versión 6.0.3
  - PyQtWebEngine = Versión 5.15.7

2.3. Vista de Procesos

2.3.1. Propósito

La vista de procesos describe el comportamiento dinámico del sistema completo como un único flujo maestro de sesión, construido a partir de las cadenas de precondición entre los casos de uso del sistema (p. ej. generar trials exige una señal previamente cargada).

El flujo de la Figura 3 se organiza en las siguientes etapas:

1. **Inicio.** El sistema registra los servicios del núcleo, incluido el `TaskOrchestrator`, y descubre los plugins disponibles a partir de sus archivos `properties.yml`.
2. **Apertura o carga.** El investigador abre un archivo de proyecto `.glab` o carga una señal desde la pantalla *Home*; la señal activa se publica en `DataStore`.
3. **Preprocesamiento.** Generación y edición de trials, selección de trials, remoción de artefactos, filtrado y ajuste de la frecuencia de muestreo.

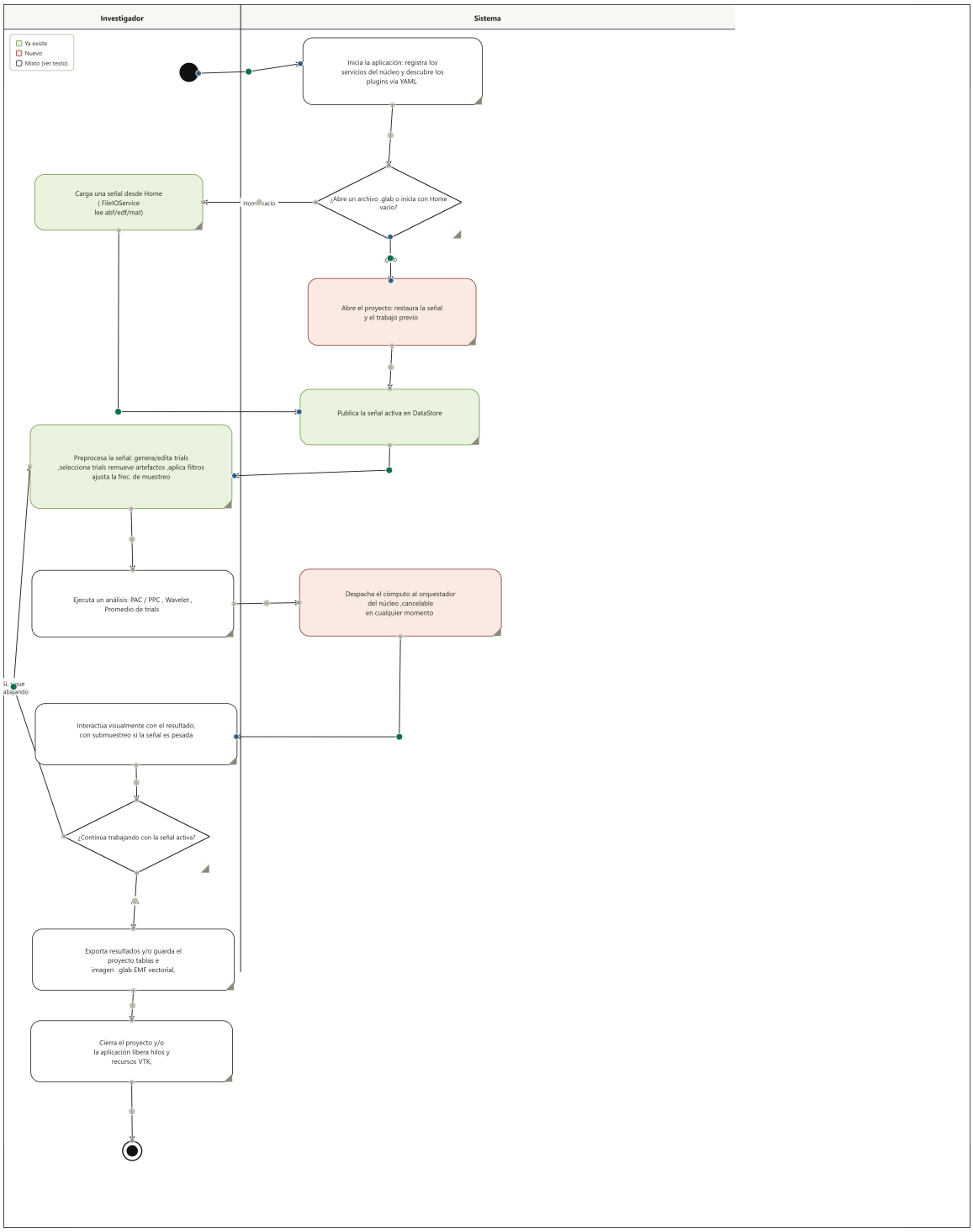


Figura 3: Flujo maestro de sesión de GammaLab 2.0, con carriles Investigador/Sistema.

4. **Análisis.** Cálculo de acoplamiento fase-amplitud (PAC) de un canal o cruzado entre canales, transformada Wavelet y promedio de trials.
5. **Orquestación.** Todo cómputo pesado se despacha al `TaskOrchestrator` del núcleo, que lo ejecuta fuera del hilo de interfaz y permite su cancelación en cualquier momento.
6. **Visualización.** Interacción visual con el resultado, con submuestreo automático cuando la señal es de gran tamaño.
7. **Iteración o cierre.** El investigador retorna al preprocesamiento para continuar el análisis, o exporta y guarda el proyecto mediante `ProjectService` antes de cerrar la sesión.

La navegación entre secciones, la limpieza de formularios y la ayuda contextual mediante `TooltipHelper` son transversales a toda la sesión: no ocupan un lugar fijo en la secuencia y se documentan como notas adjuntas al diagrama en vez de forzarse dentro del flujo principal.

2.4. Arquitectura de implementación

Las tres vistas anteriores describen el sistema a nivel de componentes. Esta sección baja al nivel en que se escribe el código: qué paquete puede importar a cuál, quién gobierna el ciclo de vida de las extensiones, en qué orden arranca y se cierra la aplicación, cómo se reparte el trabajo entre hilos, cómo se tratan los fallos y qué hay que hacer para añadir una extensión nueva.

Cada apartado se cierra con el estado real de implementación. El Software Architecture Document (SAD) prescribe trece decisiones de arquitectura (ADR-01 a ADR-13); varias están implementadas, otras lo están en parte y otras todavía no. Documentar como existente lo que no se ha construido convertiría este documento en una descripción de un sistema imaginario, así que la distinción se mantiene explícita en cada caso.

2.4.1. Organización del código y reglas de dependencia

El código se organiza en cuatro paquetes de primer nivel. La Tabla 2 indica qué componente de la Vista Lógica (Sección 2.1) vive en cada uno.

Cuadro 2: Paquetes de primer nivel y componentes que alojan

Paquete	Contenido	Componentes del SAD
`core/`	Núcleo, contrato de extensión, servicios compartidos, modelos de datos y algoritmos que cruzan fronteras de extensión.	Kernel, PluginManager, IPlugin, DataStore, FileIOService, ExportService, MeasurementService, SettingsService, ProjectService, TaskService.
`app/`	Capa de presentación: ventana principal, pantalla de bienvenida y widgets propios.	MainWindow.
`plugins/`	Las extensiones, agrupadas por categoría (`io`, `preprocessing`, `analysis`, `measure`, `faq`).	Plugins.
`test/`	Pruebas automatizadas, con la misma división por responsabilidad (`services_test`, `plugins_test`, `ui_test`, `performance_test`).	—

Dentro de `core/` la división es por responsabilidad: `core/kernel.py` (el núcleo), `core/plugins/` (contrato, descubrimiento y registro), `core/services/` (los servicios compartidos), `core/model/` (las estructuras de datos), `core/filters/` (los algoritmos compartidos) y `core/utils/` (utilidades transversales).

Reglas de dependencia. Son tres, y se enuncian en orden de importancia.

1. `core/` **no importa de** `app/` **ni de** `plugins/`**.** Es la regla que sostiene el estilo microkernel: el núcleo tiene que poder existir sin ninguna extensión concreta y sin la ventana principal. Si se invierte, el núcleo deja de ser un núcleo y pasa a ser una capa más del monolito.
2. **Las extensiones no se importan entre sí.** Lo que dos extensiones comparten sube a `core/`; lo que usa una sola se queda en su carpeta. Es la regla que hizo que `gabor.py` pasara del plugin de PAC a `core/filters/` cuando PPC empezó a necesitarlo, y la que mantiene a la wavelet de Morse dentro de PAC, porque solo PAC la usa.
3. **El cómputo no importa de Qt ni de VTK.** El módulo `compute.py` de cada extensión recibe arreglos de NumPy y devuelve arreglos de NumPy. Es lo que permite ejecutarlo en un hilo trabajador y probarlo sin levantar una interfaz gráfica (ADR-04).

Cómo se verifica. La primera regla se comprueba automáticamente: basta recorrer los archivos de `core/` y buscar importaciones de `app` o `plugins`. Al momento de redactar esta sección el resultado es **cero importaciones**, frente a 3 de `app/` hacia los otros paquetes y 32 de `plugins/`, que es la dirección correcta. La tercera regla se comprueba de la misma forma sobre cada `compute.py`, y está cubierta por pruebas: `test/services_test/test_gabor_en_core.py` verifica con el módulo `ast` que `core/filters/gabor.py` no importe nada de `plugins`, mirando las sentencias de importación y no el texto, porque los comentarios sí nombran a `plugins` y eso es documentación, no acoplamiento.

Correspondencia con las decisiones del SAD. La Tabla 3 relaciona cada decisión de arquitectura con el apartado de este documento que la desarrolla y con su estado de implementación.

Cuadro 3: Correspondencia ADR / principio → sección del SDD

ADR	Princ.	Decisión	Sección	Estado
ADR-01	P1	Estilo microkernel con núcleo propio	2.4.1, 2.4.2	Implementada
ADR-02	P1	Ciclo de vida como máquina de estados gobernada por el Kernel	2.4.2	**No implementada**
ADR-03	P2	Catálogo de eventos con carga útil tipada	2.4.2	**No implementada**
ADR-04	P3	Cómputo en un módulo sin dependencias de Qt ni VTK	2.4.1, 2.4.6	Parcial (9 de 19 extensiones)
ADR-05	P4	Descubrimiento declarativo validado	2.4.2, 2.4.6	Parcial
ADR-06	P5	`DataStore` con escritor único	2.4.4	Parcial
ADR-07	P6	Estrategia de memoria	2.4.4	Parcial
ADR-08	P7	Liberación determinista de recursos	2.4.3, 2.4.4	Parcial
ADR-09	P8	TaskOrchestrator híbrido (hilos y procesos)	2.4.4	Parcial (solo hilos, por medición)
ADR-10	P9	Control de admisión por estimación de costo	2.4.4	**No implementada**

ADR	Princ.	Decisión	Sección	Estado
ADR-11	P10	Manejador global de excepciones	2.4.5	**No implementada**
ADR-12	P11	Registro estructurado con archivo rotativo	2.4.5	**No implementada**
ADR-13	—	Python, PyQt5 y VTK como base	2.2	Implementada

2.4.2. Núcleo y ciclo de vida

Kernel. Es un `QObject` con cuatro responsabilidades y ninguna más: mantener el diccionario de extensiones registradas, mantener el diccionario de servicios, exponer un canal de eventos y recorrer las extensiones para detenerlas. No calcula, no dibuja y no lee archivos. Su interfaz pública es:

- `register_plugin(name, plugin)` — registra la extensión y le llama `initialize(kernel)`;
- `get_plugin(name)`, `get_plugins()`, `get_plugins_by_category(cat)` — consulta;
- `register_service(key, service)` / `get_service(key)` — el localizador de servicios;
- `emit_event(topic, payload)` — publicación de eventos;
- `stop_plugins()` — detención ordenada.

PluginManager. Recorre el árbol de `plugins/` buscando archivos `properties.yml`, instancia la clase declarada en cada uno y guarda el par `(meta, instancia)` en un registro indexado por el identificador de la extensión. El descubrimiento es tolerante a fallos: una carpeta cuyo descriptor esté mal formado, o cuya clase no pueda importarse, se omite con un mensaje en consola y el resto de la aplicación sigue arrancando.

Estados y transiciones. Hoy el ciclo de vida de una extensión es una secuencia, no una máquina de estados: *descubierta* (el `PluginManager` la instancia) → *inicializada* (el Kernel le llama `initialize`) → *activa* (`start` desde `MainWindow` al entrar en su pestaña, que pone el indicador `started`) → *detenida* (`stop`).

> **Brecha respecto al SAD (ADR-02).** El SAD prescribe seis estados explícitos — descubierta, inicializada, activa, inactiva, detenida y **fallida** — con las transiciones gobernadas únicamente por el núcleo. La implementación actual no mantiene el estado de cada extensión: el único indicador es el booleano `started` de `IPlugin`, las transiciones las dispara `MainWindow` y no `Kernel`, y no existe el estado *fallida*: una extensión cuya carga falla se omite en silencio y no aparece en la interfaz, en lugar de aparecer marcada como fallida. Cerrar esta brecha es un trabajo acotado — un campo de estado en el Kernel y una transición por método — pero no se ha hecho.

Registro de servicios. Los servicios se registran y se consultan por una clave de texto. Las cuatro claves vigentes son `"DataStore"`, `"FileIO"`, `"TaskService"` y `"MainWindow"`.

Brecha: las claves no están centralizadas. Hoy cada clave se escribe como literal en el punto de uso, tanto al registrar en `main.py` como al consultar en cada extensión. Un error de escritura no se detecta: `get_service()` devuelve `None` y el fallo aparece más tarde y lejos de su causa. Lo que corresponde es un módulo con las claves como constantes — por ejemplo `core/services/claves.py` — y que tanto el registro como el consumo las importen de ahí. Nótese además que la clave del servicio de entrada/salida es `"FileIO"` y no `"FileIOService"`, discrepancia que sobrevive precisamente porque no hay un único sitio donde esté escrita.

Catálogo de eventos. El canal es un único `pyqtSignal(str, object)`: un tópico de texto y una carga útil sin tipo. Los tópicos realmente en uso son cinco, y se recogen en la Tabla 4.

Cuadro 4: Eventos publicados en el canal del Kernel

Tópico	Carga útil	Significado
`signal_added`	`{"key": nombre}`	Se cargó una señal nueva.
`signal_active_changed`	nombre de la señal	Cambió la señal activa.
`trials_generated`	`{"signal": ..., çhannel": ...}`	Se recortaron trials de un canal.
`current_trial_changed`	estado del selector	El usuario cambió de trial.
`trial_discard_updated`	descarte aplicado	Se descartó o recuperó un trial.

Brecha respecto al SAD (ADR-03). El SAD prescribe un catálogo con carga útil tipada, de modo que un error de tópico se detecte al emitir. Con el canal genérico actual, publicar `"signal_add"` en lugar de `"signal_added"` no produce ningún error: simplemente no lo escucha nadie. La tabla anterior documenta el catálogo *de facto*; convertirlo en contrato exige constantes para los tópicos y una estructura declarada por carga útil.

Validaciones de carga. El descubridor exige que el `properties.yml` contenga las ocho claves obligatorias (`id`, `name`, `category`, `subcategory`, `version`, `icon`, `logic_class`, `ui_class`) y que la clase declarada en `logic_class` exista dentro de la carpeta. Lo que **no** valida, y el SAD prescribe en ADR-05:

- **Versión del contrato.** El campo `version` del descriptor es la versión de la extensión, no la del contrato `IPlugin` contra el que fue escrita. Una extensión compilada contra un `IPlugin` antiguo se carga igual y falla al usarse.
- **Dependencias.** No existe un campo para declararlas. Una extensión que necesite un servicio no registrado lo descubre en tiempo de ejecución, con un `None`.
- **Unicidad del identificador.** El registro es un diccionario indexado por `id`: dos extensiones con el mismo identificador se sobrescriben sin aviso. El Kernel sí avisa por consola de un *nombre* repetido, pero igualmente lo sobrescribe.

2.4.3. Arranque y cierre

CU-011 — Iniciar la aplicación. El orden de instanciación en `main.py` no es arbitrario; cada paso depende del anterior:

1. **Guarda del temporizador de VTK.** Antes de crear ningún widget se sustituye el manejador de temporizador de `QVTKRenderWindowInteractor` por una versión que comprueba que el widget siga visible. Va primero porque la sustitución tiene que estar puesta antes de que exista el primer interactor.
2. `QApplication`, fuentes, icono y hoja de estilos.
3. **Pantalla de bienvenida**, que informa del avance de los pasos siguientes.
4. **Kernel**.
5. **Servicios del núcleo**: `DataStore`, `FileIO` y `TaskService`. *Van antes del descubrimiento de extensiones*: `register_plugin()` llama a `initialize(kernel)`, y desde ahí una extensión ya puede pedir cualquier servicio. Si estas líneas se movieran después, recibiría `None`.
6. **Descubrimiento e instanciación** de las extensiones con `PluginManager.load_all()`.
7. **Registro en el Kernel** de cada extensión, por el nombre declarado en su descriptor.
8. `MainWindow`, que se construye con el Kernel ya poblado — necesita la lista de extensiones para montar la barra de categorías — y se registra después como servicio, porque las extensiones lo consultan para la barra de estado y los diálogos.
9. **Mostrar la ventana** en el primer ciclo del bucle de eventos.

Inyección de servicios. No hay un contenedor de inyección de dependencias: el patrón es *Service Locator*. Una extensión no recibe sus dependencias en el constructor, sino que las pide al Kernel cuando las necesita, con `kernel.get_service(clave)`. El contrato `IPlugin` ofrece envoltorios para los accesos más frecuentes — `get_datastore()`, `get_active_signal()`, `get_active_trials()`, `get_trials_de_canal()` — que resuelven el servicio y además traducen la ausencia en un aviso al usuario en lugar de en una excepción.

CU-022 — Cerrar la aplicación. El cierre recorre las extensiones registradas y llama al `stop()` de cada una. Qué libera ese `stop()` depende de la extensión; las de análisis que usan el orquestador cancelan además las tareas que tengan en vuelo con `cancel_all_from(id)`, de modo que no quede un hilo trabajador escribiendo sobre una interfaz que ya no existe.

> **Brecha respecto al SAD (ADR-08).** El SAD exige que tras el cierre «no queden hilos, procesos ni contextos gráficos activos». La detención actual no garantiza eso en un caso concreto: una tarea cancelada que no responda en dos segundos se descarta pero no se detiene, porque un hilo de Python no puede terminarse de forma segura desde fuera. El hilo desacoplado sigue vivo hasta que su función termine. Está documentado en el SAD como limitación conocida de ADR-09.

2.4.4. Hilos, tareas y recursos

Reglas de hilos para el desarrollador. Son tres y no admiten excepción:

1. **VTK y Qt solo en el hilo de interfaz.** Ninguna de las dos librerías es segura fuera de él. Un cálculo que toque un `vtkTable` desde un trabajador produce una violación de acceso, no una excepción.

2. **Ninguna extensión crea sus propios hilos.** El cómputo pesado se encola en el orquestador. Antes había extensiones con su propio `QThread`, lo que daba tantos patrones de concurrencia como extensiones y ninguna forma común de cancelar.
3. **El resultado se dibuja en el *slot* de `finished`**, que Qt ejecuta ya en el hilo de interfaz, y siempre tras comprobar que el manejador recibido sea el vigente.

TaskService. Es la implementación del TaskOrchestrator del SAD. Mantiene una cola única y ejecuta **una tarea a la vez** en un hilo trabajador. `submit(fn, owner=...)` devuelve un `TaskHandle` con cuatro señales — `progress`, `finished`, `failed` y `cancelled` — y al cómputo se le inyecta un `TaskContext` con `ctx.cancelled` y `ctx.progress(pct, mensaje)`.

Cada señal se conecta con una guarda. Al cambiar un parámetro se encola otra tarea y la anterior puede seguir viva un instante; sin la guarda `if handle is not self._handle`, su resultado sobrescribiría al nuevo:

```
handle = tasks.submit(cp.ppc_mapa, owner=self.meta.id, ...)
self._handle = handle
handle.progress.connect(partial(self._on_progress, handle))
handle.finished.connect(partial(self._on_done, handle))
handle.failed.connect(partial(self._on_failed, handle))
handle.cancelled.connect(partial(self._on_cancelled, handle))
```

Control de admisión. Existe en una forma mínima: al encolar, el servicio descarta las tareas *pendientes* de la misma extensión (`_discard_queued_from(owner)`), de modo que pulsar *Generate* cinco veces no deja cinco cálculos en cola. No existe, en cambio, la estimación de costo previa que prescribe ADR-10: el sistema no se niega a lanzar un cálculo que vaya a tardar minutos ni advierte de ello.

Cancelación. Es cooperativa: se activa un `threading.Event` y el cómputo lo consulta en sus puntos de comprobación. Si no responde en dos segundos, el servicio deja de esperarlo y descarta su resultado, pero el hilo sigue corriendo.

Política hilo/proceso. El SAD prescribe en ADR-09 un orquestador híbrido con un proceso trabajador para los cómputos aislables. **La implementación usa solo hilos**, y la razón es una medición, no un olvido: el costo de arrancar un proceso y serializar los arreglos hacia él resultó ser de **1.486 ms** frente a los **1.163 ms** que tarda el cálculo en sí. El proceso costaba más que lo que se quería paralelizar. Como las operaciones pesadas son transformadas de NumPy, que liberan el *GIL* mientras calculan, los hilos sí aprovechan varios núcleos: el reparto de trials entre cuatro hilos en PPC rinde entre **1,8**× y **2,7**× con resultado idéntico bit a bit. Lo que los hilos no dan es el aislamiento de fallos ni la cancelación forzada, que siguen siendo el argumento a favor de ADR-09 y quedan pendientes.

Política de memoria. Cuatro mecanismos, en orden de impacto:

- **Submuestreo.** Todos los análisis toman una frecuencia de muestreo objetivo y submuestrean antes de calcular. El factor es entero, así que la tasa pedida no siempre se alcanza: `tasa_efectiva()` devuelve la tasa *real* resultante, y es esa — y no la pedida — la que construye el eje de frecuencias. Tratarlas como iguales corría todo el eje hasta un 11 %.
- **Tipo numérico por etapa.** El cálculo va en `float64` y lo que se entrega a VTK para dibujar se convierte a `float32`, que basta para la pantalla y ocupa la mitad.

- **Cómputo por tandas.** Las transformadas tiempo-frecuencia se calculan en bloques de frecuencias en lugar de reservar la matriz completa: para 500 filas por 60.000 muestras la diferencia es de 480 MB a 30 MB.
- **Memoización de filtros.** El diseño de los filtros IIR está memoizado y devuelve una copia de los coeficientes, para que ningún consumidor pueda alterar la entrada compartida. Al cambiar de trial, el ahorro medido es de **7,25**×.

Liberación determinista. Al salir de una pestaña, `stop()` desactiva el interactor de VTK de la extensión. Está documentado como defecto abierto que la memoria no se libera por completo al cerrar un proyecto.

2.4.5. Fallos, registro y mensajes

Taxonomía de errores. El sistema distingue tres severidades, cada una con su canal, a través de `PluginAlerts`:

Cuadro 5: Severidades y tratamiento

Severidad	Canal	Cuándo se usa
Error	Diálogo modal (`alerts.error`)	El usuario pidió algo que no se puede hacer y tiene que enterarse: un archivo ilegible, un parámetro fuera de rango, dos canales no alineados. Interrumpe.
Aviso	Diálogo o registro (`alerts.warning`)	Se puede continuar, pero hay algo que condiciona la lectura del resultado.
Informativo	Registro y barra de estado	Avance de una tarea, resultado de una operación, condiciones del cálculo. No interrumpe.

Los avisos sobre el resultado de un cálculo no abren diálogo, y hay una razón técnica además de la de usabilidad: un modal abre un bucle de eventos anidado, y el temporizador de `QVTKRenderWindowInteractor` sigue disparando dentro de él sobre widgets que pueden estar ocultos. Tres cierres inesperados de la aplicación se rastrearon hasta ahí. Por eso las extensiones que informan de condiciones del cálculo — el índice de modulación y PPC — lo hacen con una función que devuelve una lista de textos, que el plugin registra. Un fallo de verdad sí abre diálogo, pero ese es otro camino: el *slot* de `failed`.

Captura en los trabajadores. Una excepción dentro de un cómputo no cruza el límite del hilo: el `TaskService` la captura y la convierte en la señal `failed` con su mensaje, que el *slot* correspondiente traduce en un diálogo y en la restauración del botón. El dibujo también va protegido: una excepción al renderizar se registra y se reporta, y el bloque `finally` restituye el estado de la interfaz pase lo que pase.

Archivos corruptos. `FileIOService` valida al leer y lanza una excepción con el motivo; la extensión que la invoca la traduce en un diálogo. Una carpeta de extensión con el descriptor mal formado no impide el arranque: se omite con un mensaje y el resto carga.

> **Brecha respecto al SAD (ADR-11 y ADR-12).** Dos elementos prescritos no existen en el código.

No hay manejador global de excepciones. No se instala ningún `sys.excepthook`. Una excepción no capturada en el hilo de interfaz — fuera de los bloques protegidos — termina el proceso sin que el usuario sepa por qué y sin que quede registro.

No hay registro estructurado. El sistema no usa el módulo `logging`: los mensajes salen por `print()`, a través de `IPlugin._log()`, que antepone el nombre de la extensión. No hay niveles, ni marca de tiempo, ni archivo, ni rotación. Diagnosticar un fallo que le ocurrió al usuario exige reproducirlo, que es exactamente lo que ADR-12 pretende evitar. La migración está acotada — un `logging.getLogger` por módulo y un `RotatingFileHandler` en el arranque — pero toca todas las llamadas existentes a `print`.

2.4.6. Extensibilidad

Contrato `IPlugin`. Toda extensión hereda de `IPlugin` y recibe su descriptor en el constructor. Los métodos que debe implementar son tres — `stop()`, `get_widget(parent)` y `process(data)` — y hereda `initialize(kernel)`, `start(kernel)`, el acceso a los servicios y el manejo de alertas. La Tabla 6 resume el contrato.

Cuadro 6: Contrato `IPlugin`

Método	Origen	Responsabilidad
`initialize(kernel)`	Heredado	Guardar la referencia al Kernel. Se llama al registrar.
`start(kernel)`	Heredado	Obtener `MainWindow`, suscribirse al canal de eventos y marcar `started`.
`stop()`	**Obligatorio**	Liberar lo propio: tareas en vuelo, interactor de VTK, observadores.
`get_widget(parent)`	**Obligatorio**	Construir la vista la primera vez y devolverla siempre.
`process(data)`	**Obligatorio**	Recibir datos del Kernel o de otra extensión.
`get_analysis_params()` / `apply_analysis_params(d)`	Opcional	Guardar y restaurar los parámetros con el proyecto.

Descriptor `properties.yml`. Ocho claves obligatorias: `id` (identificador único), `name` (el que se muestra y con el que se registra), `category` y `subcategory` (dónde aparece el botón; la interfaz agrupa por subcategoría sin lista fija, de modo que una subcategoría nueva aparece sola), `version`, `icon` (archivo dentro de la carpeta), `logic_class` (la clase que implementa `IPlugin`) y `ui_class`. Opcional: `description`, que alimenta la ayuda emergente.

Patrón `compute` / `ui`. Una extensión de análisis se escribe en tres archivos:

- `compute.py` — el cálculo. Recibe NumPy, devuelve NumPy. No importa Qt ni VTK. Es lo que corre en el trabajador y lo que se prueba.
- `<nombre>_plugin_ui.py` — la vista: los controles y el hueco del gráfico.
- `<nombre>_plugin.py` — la lógica de coordinación: lee los parámetros, encola, recibe y dibuja.

La separación no es estética. Permite probar el cálculo sin interfaz — las 76 pruebas del cómputo de PPC corren en segundos y sin Qt — y permite ejecutarlo en un hilo trabajador, que es lo que exige la regla de hilos. Nueve de las diecinueve extensiones siguen hoy este patrón; las de frecuencia (`fft`, `psd` y sus promedios) todavía tienen el cálculo dentro de la clase del plugin.

Puntos de extensión. Hay tres, de distinta madurez:

1. **Extensiones.** Es el punto principal y el único verdaderamente declarativo: basta una carpeta con `properties.yml` bajo `plugins/` para que el sistema la descubra al arrancar. No hay que tocar ningún registro central.
2. **Formatos de archivo y de exportación.** No son declarativos. Un formato de lectura nuevo exige un método en `FileIOService` y una rama en el punto de llamada; uno de exportación, una rama en `ExportService`. Convertirlos en puntos de extensión con registro — un diccionario de extensión de archivo a función lectora — está pendiente.
3. **Eventos.** Cualquier extensión puede publicar un tópico nuevo y cualquiera puede escucharlo, pero sin catálogo ni tipado (véase 2.4.2).

Guía para crear una extensión nueva. Los pasos, en orden:

1. Crear la carpeta bajo la categoría que corresponda, por ejemplo `plugins/analysis/connectivity/<nombre>/`.
2. Escribir `properties.yml` con las ocho claves y colocar el icono.
3. Escribir `compute.py` con el cálculo puro, y sus pruebas en `test/plugins_test/`. Conviene hacerlo antes que la interfaz: es donde están los errores que importan.
4. Escribir `<nombre>_plugin_ui.py` con los controles y el área del gráfico.
5. Escribir `<nombre>_plugin.py`: heredar de `IPlugin`, implementar `get_widget`, `stop` y `process`, encolar el cálculo con `submit()` y conectar las cuatro señales con su guarda.
6. Si el cálculo es compartido con otra extensión, subirlo a `core/filters/` en lugar de duplicarlo.
7. Verificar que la regla de dependencia se mantiene: que `compute.py` no importe Qt ni VTK, y que nada de `core/` importe la extensión nueva.

No hay que modificar `main.py`, ni el Kernel, ni `MainWindow`: el descubrimiento es automático y la agrupación por categoría también.

3. Diseño Detallado de Componentes

Esta sección describe las clases que forman GammaLab 2.0: los datos que guardan, las operaciones que ofrecen, las reglas que cumplen y los componentes con los que colaboran. El Kernel, el PluginManager, el contrato `IPlugin`, el `TaskService` y `PluginAlerts` se describieron en la Sección 2.4 y aquí solo se referencian.

Los componentes se presentan de abajo hacia arriba, de modo que cada uno se describe después de aquellos en los que se apoya. Primero va el modelo de datos, luego los servicios de soporte que operan sobre él, después los elementos de presentación que muestran la información al usuario y, por último, una vista de conjunto con el diagrama de clases completo.

3.1. Modelo de datos y almacenamiento

El modelo de datos sigue el recorrido de la información desde que se abre un archivo hasta que se guarda el trabajo:

archivo de señal → `SignalDataset` → `TrialDataset` → `DataStore` → archivo `.glab`

Un archivo de registro (`.abf`, `.edf` o `.mat`) se convierte en una señal multicanal. De un canal de esa señal se recortan trials. Las señales cargadas forman el estado de la sesión, y ese estado se guarda en disco como un proyecto. Ninguna de estas clases depende de Qt ni de VTK.

3.1.1. TrialDataset

En un experimento electrofisiológico se aplican estímulos repetidos y se registra la respuesta. Un trial es el fragmento del registro de un canal que rodea a uno de esos estímulos. Empieza t_0 segundos respecto al instante del estímulo, llamado *onset*, y termina t_1 segundos después. Como todos los trials se recortan con la misma ventana, tienen el mismo número de muestras y se pueden comparar o promediar muestra a muestra.

`TrialDataset`, definido en `core/model/trial_dataset.py`, es un *dataclass*: una clase de Python cuyo propósito es agrupar datos con tipos declarados. Una instancia no representa un trial aislado sino todos los trials de un canal, guardados en una matriz de tamaño (N_s, T). N_s es el número de muestras de cada trial y T el número de trials, de modo que cada columna de la matriz es un trial. Por ejemplo, con una frecuencia de muestreo de 10 kHz, una ventana de −0,1 a 0,5 s y 40 estímulos detectados, la matriz tiene 6000 filas y 40 columnas. Todas las columnas comparten el mismo eje de tiempo relativo. La Tabla 7 resume los campos de la clase.

Cuadro 7: Campos de `TrialDataset`

Campo	Contenido
`source`, `channel_index`, `channel_name`, `unit`	Archivo de origen, canal recortado y su unidad.
`sampling_rate`	Frecuencia de muestreo (Hz).
`t0`, `t1`, `time_rel` (N_s)	Ventana del recorte y eje de tiempo relativo.
`trials` (N_s, T)	Muestras en `float64`; una columna por trial.
`onsets_s` (T), `isi_s`	Instante de cada estímulo e intervalo entre estímulos.
`metadata`	Información sobre cómo se obtuvo el conjunto.

En los metadatos se guardan, bajo la clave `generation_params`, los parámetros exactos del recorte: canal, canal de estímulo, umbral, ventana y número de estímulos esperados. Con esos parámetros se pueden volver a recortar los mismos trials a partir de la señal, y en esa posibilidad se basa el archivo de proyecto (Sección 3.1.4).

Los trials se generan con la función de recorte de `core/filters/trials.py`, que se ejecuta como tarea del `TaskService`. Cuando la ventana de un trial se sale del registro, las muestras faltantes se rellenan con ceros para que la matriz conserve su forma rectangular. Si no se detecta ningún estímulo, el resultado es un único trial con el canal completo. Al construirse, la clase verifica que la matriz sea bidimensional y que el eje de tiempo y la lista de *onsets* tengan longitudes coherentes con ella; si alguna condición falla, la construcción se interrumpe con un error. `TrialDataset` no tiene métodos propios: las operaciones sobre los trials pertenecen a `SignalDataset`.

3.1.2. SignalDataset

`SignalDataset`, definido en `core/model/signal_dataset.py`, cumple dos funciones: representar la señal multicanal tal como se leyó del archivo y gestionar los conjuntos de trials recortados de ella. Sus instancias las crea `FileIOService` y el resto del sistema accede a ellas a través del `DataStore`.

La señal se guarda en una matriz de tamaño (C, N), con una fila por canal, acompañada del eje de tiempo y del nombre y la unidad de cada canal. El nombre del archivo, con su extensión, sirve como identificador de la señal en todo el sistema. Si la frecuencia de muestreo que trae el encabezado del archivo es incorrecta, el usuario puede corregirla: la clase recalcula el eje de tiempo, registra que la frecuencia fue modificada a mano y conserva el valor original para poder restaurarlo.

Los trials se guardan en estructuras privadas, que solo se modifican con los métodos de la clase. Todas se indexan por el par (archivo, canal), como muestra la Tabla 8.

Cuadro 8: Estructuras de trials en `SignalDataset`

Estructura	**Contenido**
Conjuntos originales	Los `TrialDataset` recortados, en orden de generación.
Descartes	Por (archivo, canal), los índices de los trials excluidos.
Conjuntos derivados	Por (archivo, canal), el resultado del preprocesamiento.
Caché	Por (archivo, canal), el último resultado de la consulta de trials activos.

Cada canal tiene un solo conjunto de trials: al recortar de nuevo un canal, el conjunto nuevo reemplaza al anterior y los de los demás canales se conservan. El canal del último conjunto recortado es el canal activo. Los análisis de un canal lo usan por defecto; los de dos canales, como PAC intercanal y PPC, piden cada canal por su nombre.

El usuario puede descartar trials y recuperarlos después. Descartar no borra datos: el trial queda marcado y se excluye al momento de consultar, como si se aplicara una máscara sobre la matriz. Los índices se refieren siempre a la numeración original del recorte.

El preprocesamiento (filtrado y modificación de artefactos) no escribe sobre los trials recortados. Trabaja sobre una copia, el conjunto derivado, que la clase acepta solo si corresponde a un conjunto original existente, tiene su misma forma y no comparte con él los datos. Desde ese momento los análisis leen el derivado, y el original sigue disponible.

Todas estas reglas confluyen en `get_active_trials`, la consulta que usan los análisis para obtener sus datos. Toma el conjunto del canal indicado, o el del canal activo si no se indica ninguno. Si existe un derivado, lo usa en su lugar. Luego excluye los trials descartados y guarda el resultado en la caché, donde se reutiliza mientras los descartes no cambien. Cualquier operación que modifica los trials elimina la información que dependía del estado anterior: recortar de nuevo un canal borra sus descartes, su derivado y su caché, y descartar un trial o registrar un derivado invalida la caché de ese canal.

3.1.3. DataStore

El `DataStore`, definido en `core/services/data_store.py`, es un almacén clave–valor en memoria que contiene el estado compartido de la sesión, empezando por las señales. Existe una única

instancia, creada al arrancar la aplicación y registrada en el Kernel, desde donde la obtienen los demás componentes.

El almacén admite un solo escritor. Al crearse guarda el identificador del hilo que lo creó, que es el hilo de interfaz, y rechaza con un error cualquier escritura que provenga de otro hilo. Por esta razón, una tarea que corre en un hilo trabajador no escribe directamente en el `DataStore`: entrega su resultado al terminar, y la escritura se realiza en el hilo de interfaz. Lecturas y escrituras toman además un candado, de modo que ningún hilo lee el almacén mientras se está modificando. Este control cubre las claves del almacén, no el interior de los objetos guardados: una consulta devuelve una referencia al objeto, por lo que una señal obtenida del almacén se modifica directamente con sus propios métodos.

El almacén no impone un esquema y acepta cualquier clave. Las que usa el sistema se resumen en la Tabla 9. Los resultados numéricos de los análisis no se guardan en el `DataStore`; permanecen en la extensión que los calculó.

Cuadro 9: Claves usadas en el `DataStore`

Clave	**Valor**
Nombre del archivo (p. ej. `23n09000.abf`)	La señal (`SignalDataset`).
`active_signal`	Clave de la señal activa.
`measurements`	Lista de mediciones registradas por `MeasurementService`.
`current_trial`	Trial en que se trabaja, mostrado en la barra de título.
`_project_service`	Referencia al `ProjectService` para componentes sin acceso al Kernel.

Cuando el usuario carga una señal nueva o cierra el proyecto, el almacén libera las señales y, con ellas, sus trials, las mediciones y el trial en curso, para que la memoria quede disponible antes de cargar la siguiente señal.

3.1.4. Archivo de proyecto `.glab`

El archivo `.glab` es la forma en disco del estado descrito en los apartados anteriores; lo escribe y lo lee `ProjectService`. Un proyecto es una carpeta que contiene el manifiesto `<nombre>.glab` en formato JSON, una copia del archivo de señal y la subcarpeta `archivos/`, destino por defecto de las exportaciones. Como la señal está copiada en la carpeta, el proyecto puede abrirse en otro equipo.

El manifiesto no contiene matrices de datos. Guarda la información necesaria para volver a producirlas a partir del archivo de señal: los trials se registran por sus parámetros de recorte, sus descartes y las recetas de preprocesamiento aplicadas, y los análisis por los parámetros configurados en cada extensión. Las mediciones son la excepción y se guardan completas, porque son pocos valores y dependen de puntos marcados a mano por el usuario. La Tabla 10 describe los campos.

Cuadro 10: Campos del manifiesto `.glab`

Campo	Contenido
`schema_version`	Versión del formato (`"1.0"`).
`project_name`, `saved_at`	Nombre del proyecto y fecha del guardado.
`signal`	Nombre del archivo, formato, frecuencia de muestreo y su origen.
`trials`	Por canal: `generation_params`, índices descartados y recetas de preprocesamiento.
`measurements`	Mediciones completas: puntos marcados, contexto y valores.
`analysis`	Parámetros de cada extensión que declara `saves_analysis: true`.

Al abrir un proyecto, el estado se reconstruye en el mismo orden en que se produjeron los datos. Primero se lee la señal y, si su frecuencia de muestreo había sido corregida, se aplica el valor guardado. Después se recortan de nuevo los trials, se reaplican los descartes y, sobre un conjunto derivado, las recetas de preprocesamiento. Por último se restauran las mediciones y se entregan sus parámetros a cada extensión. Los descartes se aplican antes que las recetas porque cada receta registra los descartes vigentes en el momento en que se aplicó.

3.1.5. Diagrama de clases del modelo de datos

La Figura 4 muestra las clases descritas en esta sección con sus atributos y operaciones. Las líneas indican que el `DataStore` almacena las señales, que cada `SignalDataset` contiene los conjuntos de trials originales y derivados de sus canales, y que el `DataStore` lanza `EscrituraNoPermitida` ante una escritura desde otro hilo.



Figura 4: Diagrama de clases del modelo de datos y almacenamiento

3.2. Servicios de soporte

Los servicios de soporte son los objetos que operan sobre el modelo de datos: leen archivos y los convierten en señales, registran mediciones, exportan resultados y guardan o abren proyectos. Se presentan de menor a mayor dependencia: el primero no depende de ningún otro componente y el último reúne a casi todos los anteriores.

3.2.1. SettingsService

`SettingsService`, definido en `core/services/settings_service.py`, guarda las preferencias de la aplicación que deben conservarse entre sesiones. Las mantiene en un diccionario en memoria y las escribe en el archivo `gamma_lab_settings.json`, en la carpeta de la aplicación. Al crearse lee el archivo; si no existe o no se puede interpretar, empieza con un diccionario vacío. Cada vez que se modifica un valor, el archivo se escribe de inmediato.

El servicio no se registra en el Kernel. Cada componente que lo necesita crea su propia instancia (`MainWindow`, el menú contextual de las gráficas y los plugins de carga de señales y de mediciones), y todas leen y escriben el mismo archivo. Las preferencias que se guardan hoy son las carpetas usadas por última vez, que el sistema propone como punto de partida en los diálogos de archivo (Tabla 11).

Cuadro 11: Preferencias guardadas por `SettingsService`

Clave	Contenido
`last_open_dir`	Última carpeta desde la que se cargó una señal.
`last_project_dir`	Última carpeta desde la que se abrió un proyecto.
`last_export_dir`	Última carpeta de exportación fuera de un proyecto.

3.2.2. FileIOService

`FileIOService`, definido en `core/services/fileio_service.py`, convierte un archivo de registro en un `SignalDataset` (Sección 3.1.2). Tiene un método de lectura por formato, y los tres entregan la señal con la misma estructura, de modo que el resto del sistema no depende del formato de origen. Se registra en el Kernel al arrancar la aplicación.

El servicio no escribe en el `DataStore` ni usa la interfaz gráfica, por lo que puede ejecutarse en un hilo trabajador. Lo invocan el plugin de carga de señales, como tarea del `TaskService`, y `ProjectService` al abrir un proyecto. Si el archivo no puede leerse o su contenido es inconsistente, el servicio lanza una excepción con el motivo, y el componente que lo invocó la presenta al usuario. La Tabla 12 resume cómo se interpreta cada formato.

Cuadro 12: Interpretación de los formatos de archivo

Formato	Biblioteca	Interpretación
ABF	pyabf	Si el archivo tiene varios barridos (*sweeps*), se toma el primero de cada canal. Todos los canales deben tener el mismo número de muestras. La frecuencia de muestreo, los nombres y las unidades se leen del encabezado.

Formato	Biblioteca	Interpretación
EDF	pyedflib	Primero se lee el encabezado de cada canal. Si todos comparten frecuencia y longitud, se leen directamente; si no, se remuestrean por interpolación lineal a la frecuencia más baja y a la duración más corta. Los canales sin nombre o sin unidad reciben valores por defecto.
MAT (clásico)	scipy	Se toma la primera variable numérica de una o dos dimensiones y se orienta con una fila por canal. Como el formato no incluye la frecuencia de muestreo, se asigna 1 Hz y se marca como valor por defecto para que el usuario la corrija.
MAT (v7.3)	h5py	Cada variable numérica se interpreta como un canal; los de menor longitud se completan con valores vacíos (NaN). La frecuencia se trata igual que en el formato clásico.

3.2.3. MeasurementService

`MeasurementService`, definido en `core/services/measurement_service.py`, permite al usuario medir sobre una gráfica marcando dos puntos con el ratón. Cada gráfica tiene su propia instancia, creada por su menú contextual. Al construirse, el servicio recibe la forma de obtener la gráfica activa y de leer y escribir la lista de mediciones del `DataStore` (Sección 3.1.3). Trabaja en el hilo de interfaz, porque opera directamente sobre los objetos de VTK.

Una medición avanza por tres estados: en reposo, esperando el primer punto y esperando el segundo. El usuario elige el tipo de medición, marca los dos puntos y el servicio vuelve al reposo; la tecla Esc cancela en cualquier momento. Cada clic se ajusta al punto de datos más cercano dentro de un radio de 10 píxeles, de modo que la medición usa valores reales de la serie y no la posición exacta del cursor. Un clic que se desplaza más de unos pocos píxeles se interpreta como arrastre y se ignora. Mientras se espera el segundo punto, el zoom y el desplazamiento quedan suspendidos para que los ejes no cambien entre un punto y otro. Los cálculos numéricos se realizan con las funciones de `core/filters/measurements.py`. La Tabla 13 resume los tipos de medición.

Cuadro 13: Tipos de medición

Tipo	Resultado
Pendiente	Pendiente entre los dos puntos, sus diferencias en X y en Y, y la distancia entre ellos.
Amplitud	Amplitud pico a pico dentro de la ventana definida por los dos puntos, con el máximo, el mínimo y sus posiciones.
Pendiente en todos los trials	Una medición de pendiente por cada trial, en las mismas posiciones; solo disponible en el plugin Trials.

Cada medición se guarda en el `DataStore` con un identificador correlativo por tipo (`slope-001`, `amplitude-002`, ...), los dos puntos, la fecha y su contexto: vista, gráfica, trial, canal, plugin y dominio (tiempo o frecuencia). Sobre la gráfica, la medición se dibuja como una línea discontinua entre los dos puntos. El contexto determina dónde se vuelve a dibujar: al cambiar de trial o de vista, el servicio borra los trazos visibles y redibuja solo las mediciones cuyo contexto coincide con el actual. El usuario puede ocultar o mostrar los trazos y eliminar la última medición, una en particular o todas. Cada cambio en la lista se notifica para que el panel de resultados se actualice.

3.2.4. ExportService

`ExportService`, definido en `core/services/export_service.py`, escribe en archivos externos el contenido de una gráfica o los resultados de un análisis. Al igual que `MeasurementService`, cada gráfica tiene su propia instancia, creada por su menú contextual. El servicio no depende de la clase del menú: recibe al construirse la forma de obtener la gráfica activa, los nombres de la señal, el canal y el plugin, y la carpeta de exportación. Con esos nombres arma el nombre propuesto para el archivo, en la forma `señal_canal_plugin_fecha.ext`.

La exportación de imágenes captura la ventana de la gráfica y la escribe como PNG, JPG, BMP o TIFF. La exportación vectorial genera un archivo EMF en el que ejes, marcas, etiquetas y barra de color son elementos vectoriales, y los mapas de calor se incrustan como imagen. Para las tablas, el sistema tiene tres caminos, que resume la Tabla 14.

Cuadro 14: Exportación de tablas

Qué se exporta	**Origen**	**Formatos**
Datos de una gráfica	Menú contextual de la gráfica. Se exportan todas las series con sus datos completos, no la versión reducida que se dibuja en pantalla.	CSV, XLSX, JSON
Resultados con varias tablas (Wavelet, PAC, índice de modulación, PPC)	El plugin arma las tablas (parámetros, resumen, matrices) y el servicio las escribe: en CSV una tras otra, en XLSX una hoja por tabla y en JSON una lista por tabla.	CSV, XLSX, JSON
Tablas de mediciones (pendiente y amplitud)	Panel de resultados. A diferencia de los dos casos anteriores, las escriben directamente los plugins de mediciones, sin pasar por `ExportService`.	CSV, XLSX

Los archivos XLSX se generan con un escritor propio (`core/utils/xlsx_writer.py`), sin bibliotecas externas. La carpeta propuesta depende de si hay un proyecto abierto: con proyecto, es su subcarpeta `archivos/` (Sección 3.1.4); sin proyecto, es la última carpeta de exportación guardada en `SettingsService`. Las exportaciones hechas dentro de un proyecto no modifican esa preferencia.

3.2.5. ProjectService

`ProjectService`, definido en `core/services/project_service.py`, guarda y abre los proyectos descritos en la Sección 3.1.4. Es el servicio con más dependencias: usa `FileIOService` para leer la señal, el `DataStore` para registrar y consultar el estado, el `TaskService` para las operaciones lentas, el algoritmo de recorte para reconstruir los trials y las extensiones para obtener y restaurar sus parámetros. Lo crea `MainWindow`, que lo registra en el Kernel y además lo guarda en el `DataStore`, para que el menú contextual de las gráficas pueda consultar la carpeta del proyecto.

El servicio mantiene el estado del proyecto abierto: su carpeta, la ruta del manifiesto, su nombre y un indicador de cambios sin guardar. El nombre por defecto se forma con el nombre del archivo de señal seguido de `_gammalab`. Las extensiones activan el indicador cuando el usuario modifica algo que forma parte del proyecto, y `MainWindow` lo consulta para pedir confirmación antes de descartar cambios. Cada vez que el proyecto se abre, se guarda, cambia de nombre o se cierra, el servicio avisa a `MainWindow` para que actualice la barra de título.

Guardar y abrir se dividen en pasos, porque una parte requiere la interfaz y otra puede tardar. La Tabla 15 muestra el reparto.

Cuadro 15: Pasos de las operaciones de proyecto

Operación	Paso	Hilo
Guardar	1. Decidir el destino. La primera vez se pide una carpeta; se rechazan la raíz del disco y las carpetas del sistema (Escritorio, Documentos, Descargas, Imágenes, Música, Videos y la carpeta del usuario).	Interfaz
	2. Copiar el archivo de señal a la carpeta, en bloques de 8 MB, con progreso y posibilidad de cancelar.	Trabajador
	3. Escribir el manifiesto con el estado actual y los parámetros de las extensiones.	Interfaz
Abrir	1. Leer el manifiesto y el archivo de señal, sin tocar el `DataStore`.	Trabajador
	2. Registrar la señal y reconstruir el estado en el orden descrito en la Sección 3.1.4.	Interfaz
Cerrar	Restablecer el estado del proyecto.	Interfaz

La copia de la señal se escribe primero en un archivo temporal, que se renombra al terminar; así, una copia cancelada o fallida nunca deja una señal incompleta en el proyecto. Si el archivo ya está en la carpeta con el mismo tamaño y la misma fecha de modificación, no se copia de nuevo. Si el guardado se cancela o falla, el proyecto conserva la ubicación que tenía antes, y si el proyecto cambió de nombre desde el último guardado, el manifiesto se renombra al guardar. Cerrar el proyecto solo restablece el estado del servicio; la liberación de las señales y de la memoria la realiza `MainWindow` a través del `DataStore` (Sección 3.1.3).

3.2.6. Diagrama de clases de los servicios de soporte

La Figura 5 muestra los cinco servicios con sus atributos y operaciones. Las cajas con la anotación «ver» corresponden a componentes descritos en otras secciones y aparecen solo para mostrar con qué se relaciona cada servicio; las líneas indican qué crea, lee o escribe cada uno.

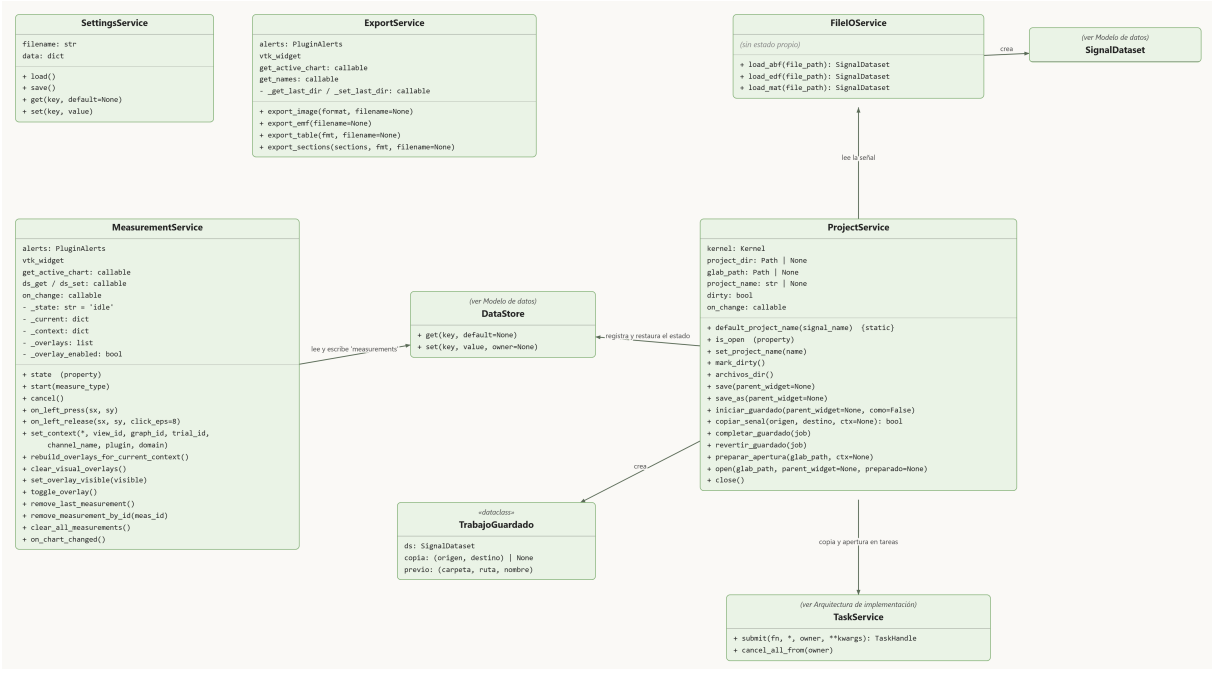


Figura 5: Diagrama de clases de los servicios de soporte

3.3. Presentación

Los elementos de presentación muestran la información al usuario y reciben sus acciones. Se presentan de las piezas al contenedor: primero las utilidades que integran VTK con Qt, que no dependen de otros componentes; después los elementos que informan al usuario del estado del sistema y el menú contextual de las gráficas, que reúne los servicios de soporte; y por último la ventana principal, que aloja a todos los anteriores y a las extensiones. Todos estos elementos trabajan en el hilo de interfaz.

3.3.1. Integración Qt/VTK

Las gráficas de GammaLab se dibujan con VTK dentro de widgets de Qt. Cada extensión crea su propio widget VTK, y la ventana principal conserva esos widgets al cambiar de módulo para no perder el trabajo del usuario. Esta combinación se apoya en un conjunto de utilidades, ubicadas en `core/utils/`, que resuelven la conversión de datos, el dibujo de series largas, la interacción con el ratón y el manejo de los recursos de las gráficas que no están en pantalla. La Tabla 16 las resume.

Cuadro 16: Utilidades de integración Qt/VTK

Módulo	Función
`vtk_temporizador`	Agrega una comprobación al temporizador interno del widget VTK de Qt, para que no actúe sobre un widget oculto o destruido.
`adapters`	Convierte una señal o una matriz de trials en la tabla de columnas que VTK usa para dibujar.
`submuestreo`	Reduce las series largas a los puntos que se dibujan en pantalla.
`chart_coords`, `cursor_readout`	Convierten píxeles de pantalla en coordenadas de los ejes y muestran la posición del cursor.
`vtk_observers`	Registra los observadores de eventos de cada gráfica para poder retirarlos y reponerlos.
`vtk_ventanas`	Libera la ventana OpenGL de un módulo que deja de estar en pantalla.

El widget VTK de Qt incluye un temporizador que se ejecuta cada 10 ms sin comprobar el estado del widget. La utilidad del temporizador reemplaza, una sola vez al arrancar la aplicación, el método que atiende ese temporizador por una versión que no actúa si el widget no está visible, no tiene ventana o ya fue destruido. Como el reemplazo se hace sobre la clase, cubre los widgets de todas las extensiones, incluidas las que se agreguen en el futuro.

El adaptador de datos construye una tabla con una columna de tiempo y una columna por canal o por trial; la de la señal se guarda en el propio `SignalDataset` (Sección 3.1.2) y se reutiliza mientras la señal no cambie. El submuestreo limita cada serie a unos 2000 puntos dibujados: divide la serie en tramos y conserva el mínimo y el máximo de cada uno, de modo que los picos siguen visibles. Al acercar la gráfica, la serie se reduce de nuevo sobre el rango visible y recupera el detalle hasta la resolución original. Solo se reduce lo que se dibuja: los datos completos se conservan, y son los que leen la exportación y las mediciones (Sección 3.2).

La lectura del cursor muestra la posición del ratón en unidades de los ejes, en una esquina de la gráfica y en la barra de estado; es la posición libre del cursor, no el punto de datos más cercano. Los observadores de eventos (movimiento del ratón, clics) se registran a través de una

utilidad que los recuerda por widget, de modo que la ventana principal pueda retirarlos cuando el usuario sale de un módulo y reponerlos cuando vuelve. Por último, la ventana OpenGL de un módulo oculto puede liberarse: se descartan su contexto gráfico y su memoria, pero se conservan el widget, la escena y los resultados, y VTK vuelve a crear la ventana la próxima vez que el módulo se dibuja.

3.3.2. Retroalimentación al usuario

Los mensajes al usuario se emiten a través de `PluginAlerts` (Sección 2.4) con tres severidades, cada una con su propio canal de presentación, como muestra la Tabla 17.

Cuadro 17: Presentación de los mensajes por severidad

Severidad	**Presentación**
Información	Tarjeta azul no modal que se cierra sola a los pocos segundos.
Advertencia	Tarjeta ámbar no modal que permanece hasta que el usuario la cierra.
Error	Diálogo modal, porque interrumpe la operación en curso.

Las tarjetas no modales las administra `NotificationCenter` (`core/utils/notifications.py`), que las apila en la esquina inferior izquierda del área de trabajo, con un máximo de cuatro visibles, y no repite un mismo aviso si se emite varias veces seguidas. Cada tarjeta es una ventana nativa, porque las gráficas de VTK también lo son y cubrirían a un widget común.

El avance de las tareas lo muestra `TaskProgressBar` (`core/utils/task_progress.py`), una barra única ubicada en la barra de estado de la ventana principal. Está conectada al `TaskService` y aparece automáticamente cada vez que una extensión encola un cálculo, con el porcentaje, un mensaje y un botón para cancelar, sin que la extensión tenga que hacer nada. La barra de estado muestra además mensajes breves coloreados según su severidad y la lectura del cursor descrita en la Sección 3.3.1.

3.3.3. VTKContextMenu

`VTKContextMenu`, definido en `core/utils/vtk_context_menu.py`, es el menú que aparece al hacer clic derecho sobre una gráfica. Cada gráfica tiene su propia instancia, creada por la extensión al dibujar. Al construirse recibe la gráfica, su widget y los nombres de la señal, el canal y la extensión, y crea a su vez un `ExportService` y un `MeasurementService` para esa gráfica, además de su propia instancia de `SettingsService`. Su función es reunir en un solo lugar las acciones comunes a todas las gráficas y delegarlas en el servicio que corresponde. La Tabla 18 muestra el contenido del menú.

Cuadro 18: Contenido del menú contextual de las gráficas

Submenú	**Acciones**	**Delega en**
Zoom	Zoom horizontal, vertical o en ambos ejes; restablecer la vista.	El propio menú
Measurements	Pendiente, amplitud, pendiente en todos los trials; cancelar; mostrar u ocultar trazos; eliminar la última medición o todas.	`MeasurementService`

Submenú	Acciones	Delega en
Export as image	PNG, JPG, BMP, TIFF y EMF vectorial.	`ExportService`
Export table	CSV, JSON y XLSX.	`ExportService`

Además del menú, el componente configura el zoom con la rueda del ratón (sobre los dos ejes, o sobre uno solo si se mantiene pulsada la tecla Ctrl o Shift), conecta la lectura del cursor y reenvía los clics de la gráfica al servicio de mediciones. Las extensiones pueden agregar acciones propias al menú. También proporciona al servicio de exportación la carpeta de destino descrita en la Sección 3.2.4, que consulta en el `ProjectService` guardado en el `DataStore`.

3.3.4. MainWindow

`MainWindow`, definida en `app/view/main_window.py`, es la ventana principal de la aplicación y el contenedor de todos los elementos anteriores y de las extensiones. Se construye después de registrar las extensiones, porque necesita conocerlas para armar la navegación, y se registra en el Kernel como servicio. Su estructura visual se define en `Ui_MainWindow` (`main_window_ui.py`), una clase generada a partir del diseño de la interfaz y sin lógica propia. Al construirse, la ventana crea el `ProjectService`, el centro de notificaciones y la barra de progreso de tareas. Sus responsabilidades se agrupan como muestra la Tabla 19.

Cuadro 19: Responsabilidades de `MainWindow`

Grupo	Responsabilidad
Navegación	Secciones Home, Preprocessing y Analysis; los botones de cada sección se agrupan por subcategoría y se ordenan según el `properties.yml` de cada extensión. Incluye una barra de búsqueda con autocompletado sobre los nombres de las extensiones.
Área de trabajo	Muestra la vista de la extensión seleccionada y administra el cambio entre módulos.
Proyecto	Menú Archivo: cargar señal, guardar, guardar como, abrir y cerrar proyecto, coordinando `ProjectService` y `TaskService`.
Barra de título	Nombre de la señal activa, nombre del proyecto (editable con doble clic) e indicador del trial en uso.
Barra de estado	Mensajes por severidad, barra de progreso y lectura del cursor.
Panel de resultados	Tablas de mediciones de pendiente y amplitud, con exportación a CSV y XLSX.
Explorador	Vista de la carpeta del proyecto abierto, con operaciones básicas sobre archivos.
Cierre	Confirmación de cambios sin guardar y cierre ordenado de la aplicación.

El cambio entre módulos sigue siempre la misma secuencia. Al salir de un módulo, la ventana cancela sus tareas en el `TaskService`, lo detiene, retira los observadores de sus gráficas y oculta su vista; a continuación, salvo que el módulo lo desactive en su `properties.yml`, libera su ventana OpenGL una vez mostrado el módulo siguiente. Al entrar en un módulo, la ventana lo inicia la primera vez, construye su vista o reutiliza la existente, repone sus observadores y la muestra. Las vistas se conservan durante toda la sesión, de modo que el usuario encuentra cada módulo como lo dejó.

La barra de título refleja el estado de la sesión. El nombre del proyecto usa `EditableLabel` (`app/view/widgets/editable_label.py`), una etiqueta que se convierte en campo de texto al hacer doble clic y comunica el nuevo nombre al `ProjectService`. El indicador de trial muestra con qué trials trabaja el módulo visible, numerados por su posición entre los válidos; se actualiza cada vez que una extensión publica un cambio y cambia de color cuando el trial mostrado está descartado. Cuando no hay ningún módulo activo, la sección Home muestra un panel de bienvenida, y el área de trabajo muestra el logotipo de la aplicación mientras no haya una señal cargada.

Al cargar una señal nueva o cerrar el proyecto, la ventana libera la señal anterior: cancela las tareas en curso, destruye las vistas de los módulos, pide a cada extensión que suelte sus referencias a la señal y libera las señales del `DataStore` (Sección 3.1.3). Al cerrar la aplicación, pide confirmación si hay cambios sin guardar, cancela todas las tareas, detiene las extensiones y finaliza las ventanas OpenGL de todos los módulos antes de salir. Durante el arranque, antes de que exista la ventana principal, `SplashScreen` (`app/view/splash_screen.py`) muestra una imagen con una barra de progreso y el paso en curso, según la secuencia descrita en la Sección 2.4.

3.3.5. Diagrama de clases de la presentación

La Figura 6 muestra los elementos de presentación con sus atributos y operaciones. `MainWindow` ocupa el centro porque crea o aloja a casi todos los demás; las cajas con la anotación «ver» remiten a componentes descritos en otras secciones.

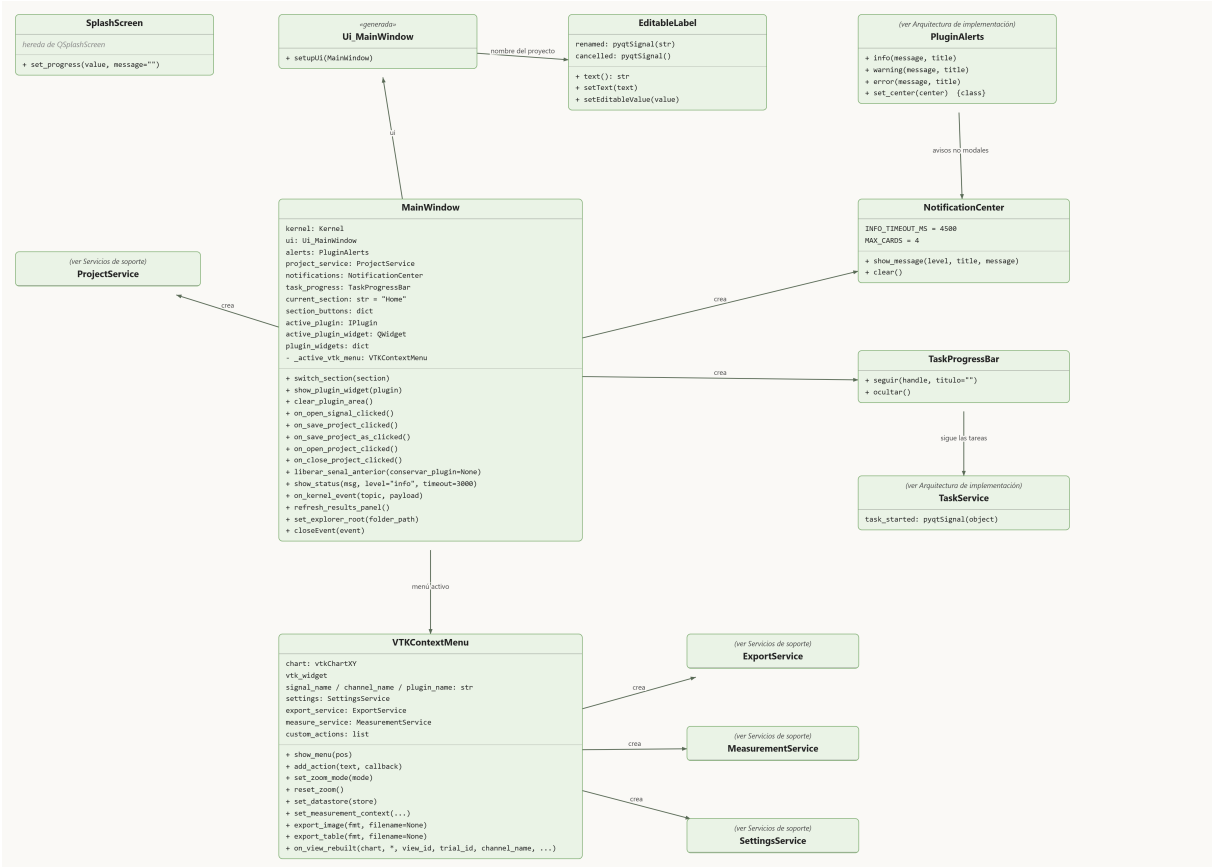


Figura 6: Diagrama de clases de los elementos de presentación

4. Comportamiento del sistema

4.1. Diagrama abrir proyecto

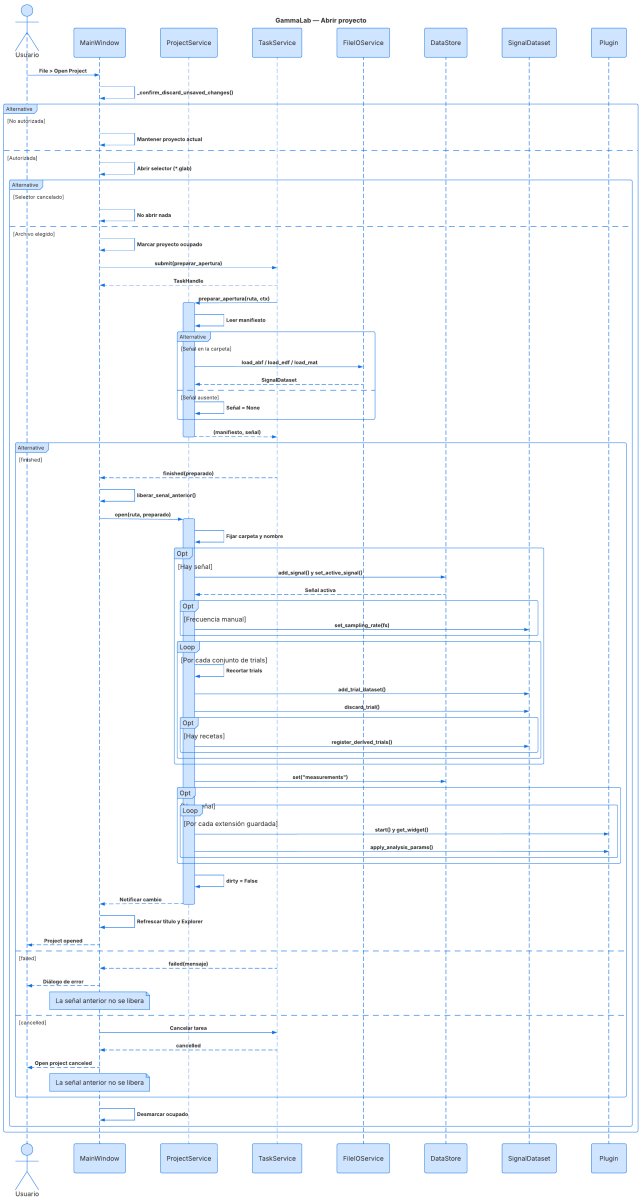


Figura 7: Diagrama de secuencia para abrir proyecto

Este diagrama representa el flujo que sigue GammaLab 2.0 cuando el usuario abre un proyecto desde la opción **File >Open Project**. El proceso comienza en `MainWindow`, que invoca `_confirm_discard_unsaved_changes()` para verificar si es posible abandonar el proyecto actual. Es el mismo mecanismo de confirmación descrito en el diagrama de cerrar proyecto: si hay un cálculo en curso o cambios sin guardar, se consulta al usuario, y si la apertura no se autoriza, el proyecto actual se mantiene sin cambios.

Cuando la apertura se autoriza, `MainWindow` muestra el selector de archivos filtrado por la extensión `.glab`. Si el usuario lo cierra sin elegir un archivo, no se abre nada. Si elige uno, `MainWindow` marca el proyecto como ocupado y envía la lectura a `TaskService` mediante `submit(preparar_`

apertura). El servicio devuelve inmediatamente un TaskHandle y ejecuta el trabajo en un hilo trabajador, de modo que la interfaz permanece disponible y el usuario puede seguir el avance en la barra de progreso.

Dentro de la tarea, ProjectService ejecuta preparar_apertura(), que lee el manifiesto del proyecto y busca el archivo de señal en la carpeta. Si lo encuentra, lo carga mediante FileIOService con el método que corresponde a su formato (load_abf, load_edf o load_mat) y obtiene un SignalDataset; si no lo encuentra, la señal queda vacía. En esta etapa no se modifica el DataStore ni la interfaz: la tarea solo devuelve el manifiesto y la señal leída.

Al terminar, TaskService comunica el resultado a MainWindow mediante una de tres señales. Cuando la lectura finaliza correctamente (finished), MainWindow ejecuta liberar_senal_anterior(), el mismo procedimiento descrito en el diagrama de cerrar proyecto, para cancelar las tareas pendientes, destruir los widgets de los plugins y liberar las señales del proyecto anterior. A continuación llama a open() en ProjectService, que fija la carpeta y el nombre del proyecto y reconstruye su estado en el hilo de interfaz.

Si el proyecto tiene una señal, ProjectService la registra en el DataStore mediante add_signal() y la fija como señal activa con set_active_signal(). Si la frecuencia de muestreo había sido corregida a mano, la aplica con set_sampling_rate(). Después recorre los conjuntos de trials guardados: vuelve a recortar cada uno con sus parámetros, lo agrega a la señal con add_trial_dataset(), reaplica los descartes con discard_trial() y, si había recetas de preprocesamiento, registra el conjunto derivado con register_derived_trials(). Luego restaura las mediciones en el DataStore y, por cada extensión guardada, inicia el plugin, construye su widget y le entrega sus parámetros mediante apply_analysis_params().

Finalmente, ProjectService establece dirty = False y notifica a MainWindow, que actualiza el título de la ventana y el Explorer e informa al usuario con el mensaje **"Project opened"**.

Si la lectura falla (failed), MainWindow muestra un diálogo con el error; si el usuario la cancela desde la barra de progreso, TaskService confirma la cancelación (cancelled) y la interfaz muestra el mensaje **"Open project canceled"**. En ambos casos no se ejecuta liberar_senal_anterior(), por lo que el proyecto que estaba abierto se conserva intacto. Sea cual sea el resultado, MainWindow desmarca el proyecto como ocupado al terminar. De esta manera, el diagrama muestra que el estado anterior solo se reemplaza cuando la lectura del nuevo proyecto ha terminado correctamente.

4.2. Diagrama guardar proyecto

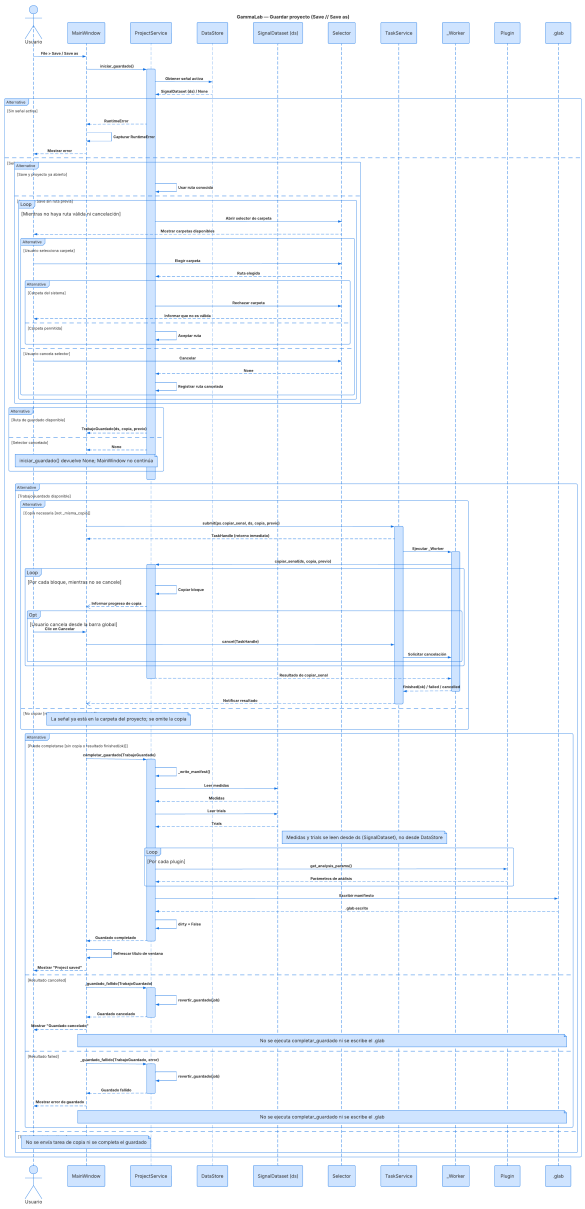


Figura 8: Diagrama de secuencia para guardar proyecto

Este diagrama representa el flujo que sigue Gamma Lab v.20 al momento de usar `save` y `save as` en un proyecto. El proceso comienza cuando el usuario solicita guardar el poryecto desde la interfaz pirncipal `MainWindow`, que llama al método `inicar_guardado` de `ProjectService`.Este servicio verifica que exista una señal activa mediante `DataStore`. Si no hay una señal disponible, se genera un `RuntimeError`, que es capturado por `MainWindow` para informar al usuario y detener el proceso.

Cuando existe una señal cargada, `ProjectService` determina la ruta en la cual se guardará el proyecto. Si la ruta es válida, se utiliza directamente; de lo contrario, se abre el selector de carpetas para que el usuario elija una ubicación. El sistema valida la carpeta seleccionada y, si no está permitida, informa al usuario y permite realizar una nueva selección. Este ciclo se mantiene hasta que la carpeta seleccionada sea válida o el usuario cancele la selección. En este último caso, `iniciar_guardado` devuelve `None` y `MainWindow` cancela la acción.

Una vez definida la ruta, `ProjectService` devuelve la información necesaria para realizar el guardado. Si la señal requiere ser copiada a la carpeta del proyecto, `MainWindow` envía la tarea a `TaskService`, que ejecuta el trabajo de forma asíncrona mediante `_Worker`. La copia se realiza por bloques y se informa el progreso al usuario. Durante esta etapa, el usuario puede cancelar le proceso desde la barra de progreso global; la solicitud se transmite a `TaskService` y al trabajador encargado de la operación. En caso que la señal ya se encuentra en la carpeta del proyecto, se omite esta tarea de copia.

Cuando la copia finaliza correctamente, o cuando no era necesaria, `MainWindow` llama al método `completar_guardado()` dentro de `ProjectService`. Allí se genera el manifiesto del proyecto, se consultan las medidas y los trials directamente desde `SignalDataset`, y se recorren los plugins para obtener sus parámetros de análisis mediante `get_analysis_params()`. Finalmente, se escribe el archivo `.glab`, se establece `dirty = False` y se notifica a `MainWindow` que el guardado se completó. La interfaz actualiza el título de la ventana y muestra el mensaje de confirmación **"Project saved"**.

En caso contrario, si la copia es cancelada o falla, se ejecuta `guardado_fallido()`, que revierte el estado temporal del guardado y comunica el resultado correspondiente al usuario. En estos casos no se ejecuta `completar_guardado()` ni se escribe el archivo `.glab`. De esta manera, el diagrama muestra que la generación del archivo del proyecto depende de que todas las operaciones previas finalizen correctamente.

4.3. Diagrama cerrar proyecto

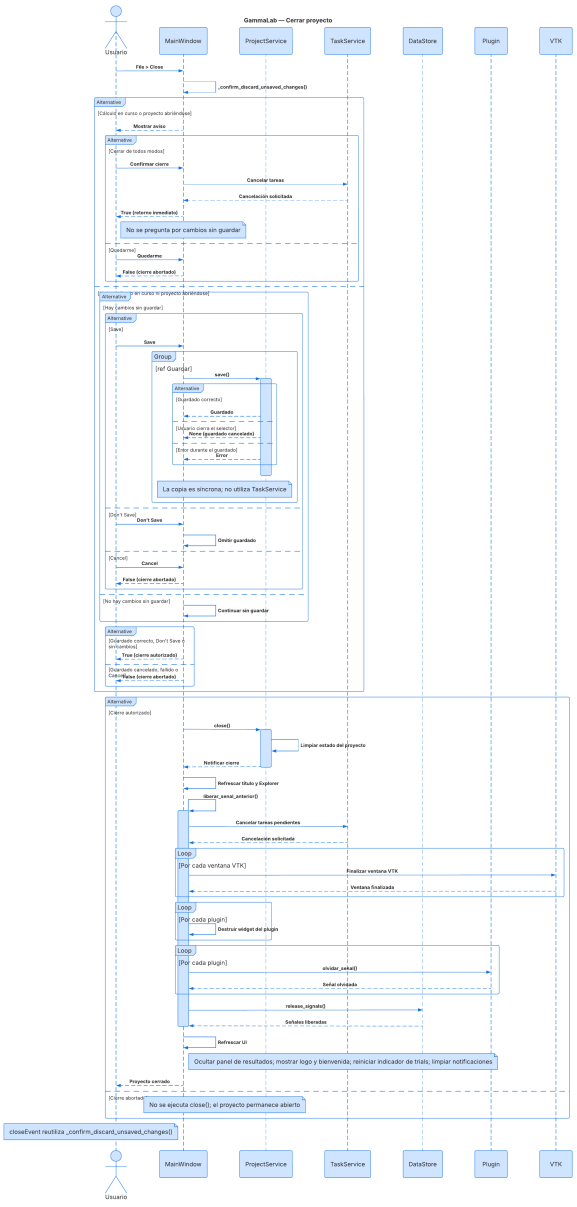


Figura 9: Diagrama de secuencia para cerrar proyecto

Este diagrama representa el flujo que sigue Gamma Lab v.20 al momento que un usuario solicita cerrar un proyecto desde la opción **File >Close**. El proceso comienza en `MainWindow`, que invoca `_confirm_discard_unsaved_changes()` para determinar si el cierre puede realizarse de forma segura y si es necesario solicitar una confirmación al usuario.

En primer lugar, se verifica si hay un cálculo en ejecución o si el proyecto se encuentra en proceso de apertura. Cuando se presenta alguna de estas condiciones, la interfaz muestra un aviso que permite al usuario decidir entre cerrar de todos modos o permanecer en el proyecto. Si el usuario confirma el cierre, `MainWindow` solicita a `TaskService` la cancelación de las tareas y autoriza inmediatamente el cierre. En esta situación no se realiza una consulta adicional sobre los cambios sin guardar. Si el usuario decide quedarse, el método devuelve `False` y el cierre se interrumpe.

Cuando no hay cálculos en ejecución ni procesos de apertura, el sistema verifica si existen cambios sin guardar. Si los hay, presenta las opciones **Save**, **Don't Save** y **Cancel**. Al seleccionar **Save**, `MainWindow` solicita a `ProjectService` ejecutar `save()`. Este guardado se realiza de manera síncrona y no utiliza `TaskService`. Si el guardado termina correctamente, el flujo puede continuar; si el usuario cancela el selector de guardado o se produce un error, el cierre se aborta. Al seleccionar **Don't Save**, se omite el guardado y se permite continuar. En cambio, **Cancel** interrumpe el procedimiento y mantiene el proyecto abierto. Si no existen cambios pendientes, el sistema también puede continuar sin guardar.

Una vez autorizada la operación, `MainWindow` llama a `close()` en `ProjectService`, que limpia el estado del proyecto y notifica el cierre a la interfaz. Posteriormente, se actualizan el título de la ventana y el Explorer. A continuación, `MainWindow` ejecuta `liberar_senal_anterior()` para liberar los recursos asociados a la señal y al proyecto. Este procedimiento comienza solicitando la cancelación de las tareas pendientes mediante `TaskService`. Después, se recorren las ventanas de `VTK` para finalizarlas, se destruyen los widgets de los plugins y se llama a `olvidar_senal()` en cada plugin para desvincularlo de la señal anterior. Finalmente, `DataStore` ejecuta `release_signals()` para liberar las señales almacenadas.

Como última etapa, `MainWindow` actualiza la interfaz: oculta el panel de resultados, muestra el logo y la pantalla de bienvenida, reinicia el indicador de trials y limpia las notificaciones. Una vez terminadas estas operaciones, informa al usuario que el proyecto se ha cerrado.

Si en cualquiera de las validaciones iniciales el cierre es cancelado o no puede completarse, no se ejecuta `ProjectService.close()` y el proyecto permanece abierto. El diagrama también muestra que el evento `closeEvent` reutiliza `_confirm_discard_unsaved_changes()`, manteniendo el mismo mecanismo de confirmación cuando el usuario intenta cerrar la ventana principal.

4.4. Diagrama wavelet

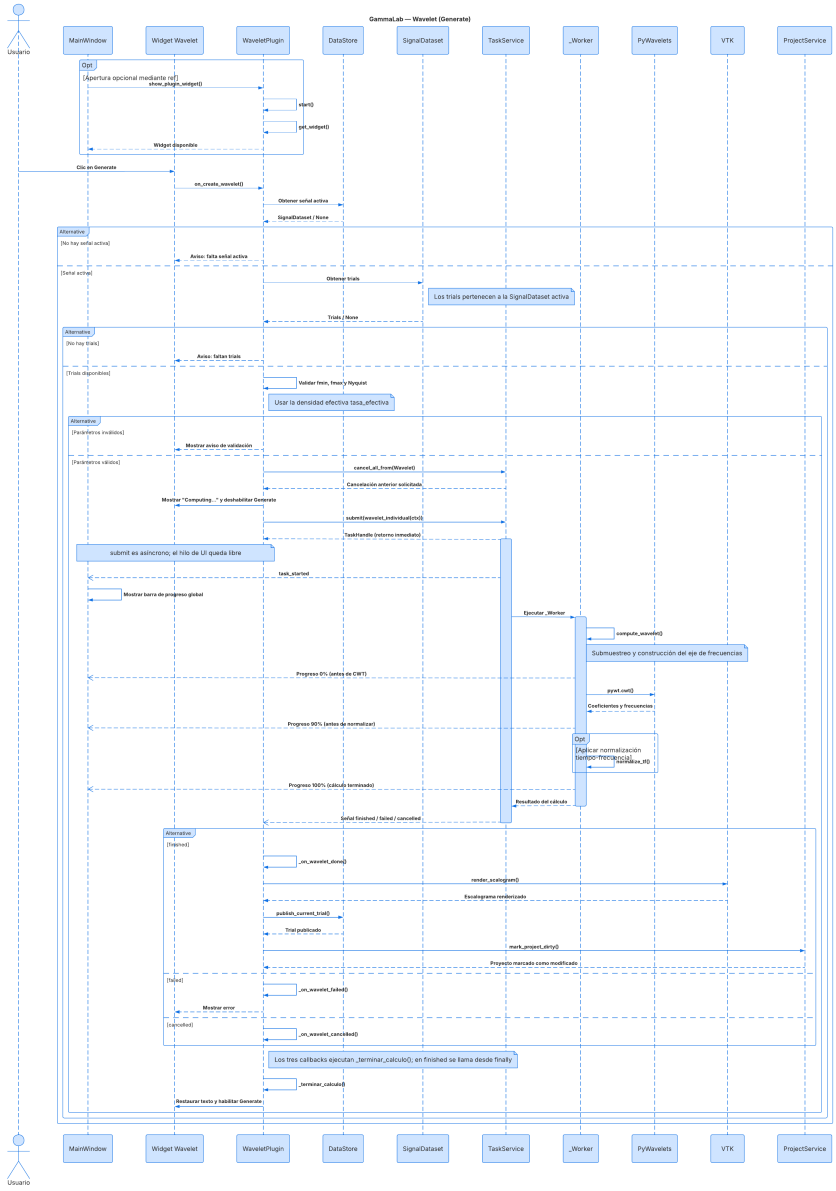


Figura 10: Diagrama de secuencia para wavelet

Este diagrama representa el flujo que sigue Gamma Lab v.20 al momentode generar un análisis con Wavelet a partir de los trial de una señal activa. Este flujo comienza con la apertura del widget de wavelet desde `MainWindow`. Para ello, se solicita al `WaveletPlugin` que inicialice el plugin y proporcione su interfaz mediante `get_widget()`. Una vez disponible el widget, el usuario puede seleccionar los parámetros del análisis y ejecutar la opción **Generate**.

Al iniciar la generación, el widget invoca `on_create_wavelet()` en `WaveletPlugin`. El plugin consulta la señal activa y verifica que exista un objeto `SignalDataset`. Si no hay una señal activa, se muestra una advertencia y el proceso se detiene. Si existe, se obtienen sus trials directamente desde `SignalDataset`, ya que estos pertenecen a la señal activa. Cuando no hay trials disponibles, también se informa al usuario y no se continúa con el cálculo.

Si existen trials, el plugin valida los parámetros de entrada, incluyendo los límites de frecuencia (`fmin` y `fmax`) y la frecuencia de Nyquist. Para esta validación se utiliza la densidad de muestreo

efectiva (`tasa_efectiva`). Si los parámetros no son válidos, se muestra el aviso correspondiente. En caso contrario, el plugin solicita la cancelación de cálculos anteriores relacionados con Wavelet mediante `TaskService`, cambia el estado del widget a **"Computing..."** y deshabilita el botón **Generate** para evitar que se inicie otra ejecución mientras el cálculo está en curso.

A continuación, `WaveletPlugin` envía el cálculo a `TaskService` mediante `submit (wavelet_individual (ctx))`. Este servicio devuelve inmediatamente un `TaskHandle` y ejecuta el trabajo de forma asíncrona a través de `_Worker`, permitiendo que el hilo de la interfaz permanezca libre. El trabajador realiza el cálculo Wavelet mediante `compute_wavelet ()`, que utiliza PyWavelets (`pywt.cwt`) para obtener los coeficientes y las frecuencias correspondientes. De acuerdo con la configuración del análisis, también puede aplicarse una normalización en el dominio tiempo-frecuencia mediante `normalize_tf ()`.

Durante la ejecución se actualiza el progreso global: se informa un avance inicial antes de ejecutar la transformada, un avance aproximado del 90 % antes de la normalización y el 100 % cuando el cálculo termina. Al finalizar, `TaskService` comunica el resultado al plugin mediante una de las señales disponibles: `finished`, `failed` o `cancelled`.

Cuando el cálculo finaliza correctamente, se ejecuta `on_wavelet_done()`. El plugin solicita a `VTK` la representación del escalograma mediante `render_scalogram()`, publica el trial generado a través de `publish_current_trial()` y notifica a `ProjectService` que el proyecto ha sido modificado mediante `mark_project_dirty()`. Si ocurre un error, `on_wavelet_failed()` se encarga de mostrar el mensaje correspondiente; si el usuario cancela la operación, `on_wavelet_cancelled()` gestiona ese resultado.

Finalmente, los tres callbacks ejecutan `_terminar_calculo()` para restaurar el estado del widget, recuperar el texto original del botón y habilitar nuevamente la opción **Generate**. Así, el diagrama representa tanto el procesamiento exitoso como las rutas de validación, error y cancelación, mostrando la interacción entre la interfaz, el plugin, el sistema de tareas, las herramientas de procesamiento y la visualización.

5. Interfaz de usuario

Esta sección presenta la interfaz de GammaLab 2.0 tal como la ve el usuario. Primero se describe la organización de la pantalla principal y sus elementos; después, las pestañas de la barra de navegación en el orden en que suelen usarse: *File*, para cargar señales y gestionar proyectos; *Preprocessing*, para dividir la señal en trials y prepararlos; y *Analysis*, con los análisis disponibles. Cada elemento se acompaña de una captura de la aplicación. Los componentes de código que implementan estos elementos se describen en la Sección 3.3.

5.1. Organización de la pantalla principal

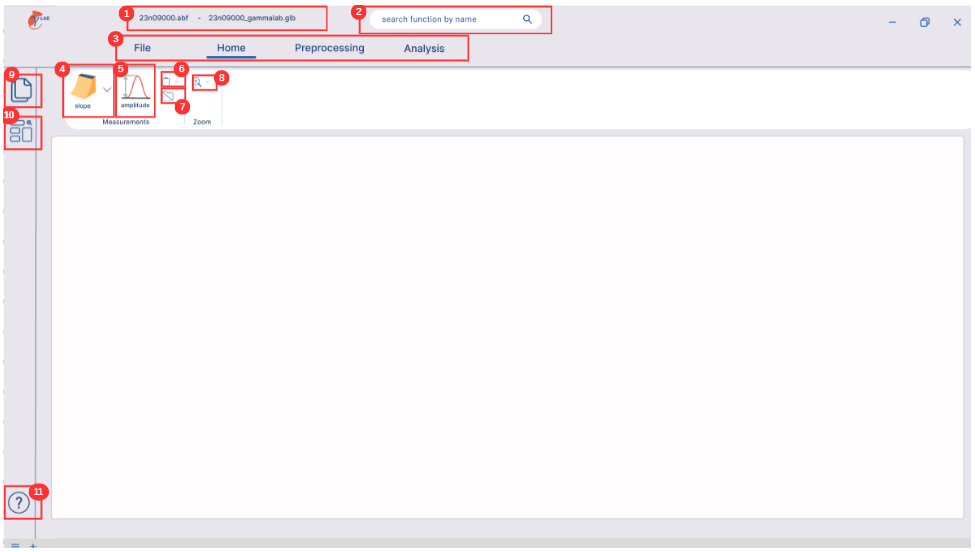


Figura 11: Organización de la pantalla principal de Gamma Lab

La pantalla principal se compone de los siguientes elementos, numerados en la Figura 11:

- **1 - Panel de control:** muestra el nombre de la señal abierta junto al nombre, editable, del proyecto activo, guardado con extensión `.glab`.
- **2 - Barra de búsqueda:** ubicada en la parte superior, permite localizar y acceder rápidamente a una funcionalidad específica del sistema sin tener que recorrer manualmente la barra de navegación.
- **3 - Barra de navegación:** agrupa las pestañas *File*, *Home*, *Preprocessing* y *Analysis*, permitiendo alternar entre la gestión de archivos, las mediciones sobre la señal, el preprocesamiento y los módulos de análisis.
- **4 - slope:** opción de la categoría *Measurements* (pestaña *Home*) que permite medir la pendiente de la señal entre dos puntos seleccionados por el usuario, mediante las subopciones *Slope 2 Trials* (calculada sobre dos ensayos) y *Slope All Trials* (calculada sobre la totalidad de los ensayos generados).

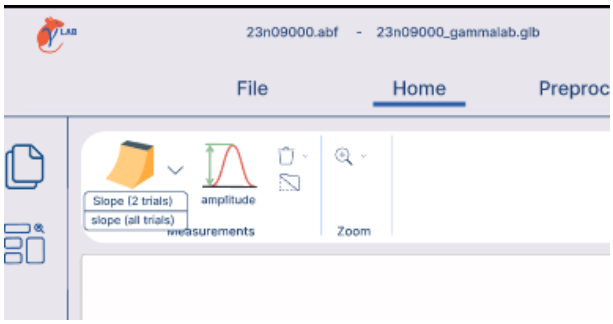


Figura 12: opciones de slope

- **5 - amplitude:** opción de la categoría *Measurements* (pestaña *Home*) que permite medir la amplitud de la señal entre dos puntos seleccionados por el usuario sobre la gráfica.
- **6 - eliminar medida:** permite eliminar la última medición de *slope* o *amplitude* realizada, o bien la totalidad de las mediciones registradas sobre la señal.

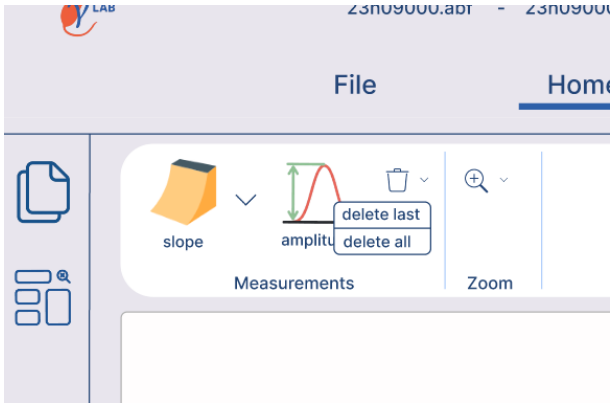


Figura 13: opciones de eliminar medida

- **7 - mostrar/ocultar:** permite mostrar u ocultar en pantalla las mediciones de *slope* y *amplitude* realizadas previamente sobre la señal, sin necesidad de eliminarlas.
- **8 - zoom:** activa o desactiva la función de acercamiento sobre la gráfica de la señal, permitiendo observar con mayor detalle un segmento de la señal y las mediciones realizadas sobre él.



Figura 14: opciones de zoom

- **9 - explorador de archivos:** despliega, desde la barra lateral izquierda, la carpeta del

sistema de archivos en la que se encuentra almacenado el proyecto actual junto con las señales y resultados exportados, sin salir de la aplicación.

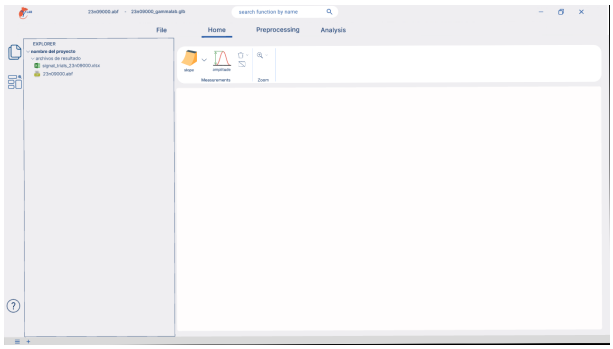


Figura 15: explorador de archivos desplegado

- **10 - resultados:** despliega, desde la barra lateral izquierda, una ventana inferior con los resultados numéricos de las mediciones de *slope* y *amplitude* realizadas sobre la señal, los cuales pueden exportarse en formato `.csv` para su análisis externo.

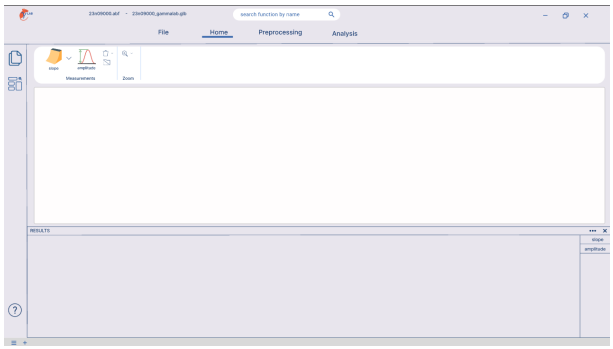


Figura 16: resultados desplegados

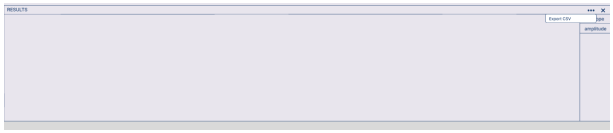


Figura 17: Los resultados pueden ser exportados en formato csv.

- **11 - ayuda:** despliega, desde la barra lateral izquierda, el manual de usuario del sistema, con información de referencia sobre el uso de cada funcionalidad.

5.2. Carga y guardado de archivos (pestaña *File*)

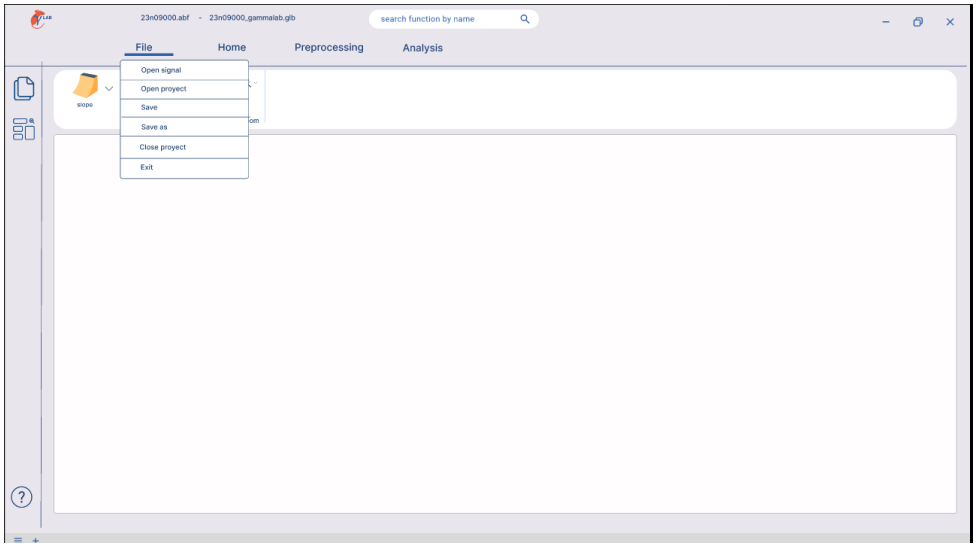


Figura 18: Pestaña File de Gamma Lab

La pestaña *File* permite gestionar la carga y el guardado de señales y proyectos. Agrupa las siguientes opciones:

- **Open Signal:** permite cargar una señal individual dentro de la aplicación para su preprocesamiento y análisis.
- **Open Project:** permite abrir un proyecto previamente guardado con extensión `.glab`, restaurando la señal activa, los trials, los análisis y los parámetros de la sesión.
- **Save:** guarda los cambios realizados sobre el proyecto actualmente abierto.
- **Save As:** permite guardar el proyecto actual bajo un nuevo nombre o ubicación, sin sobrescribir el archivo original.
- **Close Project:** cierra el proyecto que se encuentra abierto, liberando la señal activa de memoria.
- **Exit:** cierra la aplicación.

A continuación se ilustra el flujo de carga de una señal y de un proyecto:

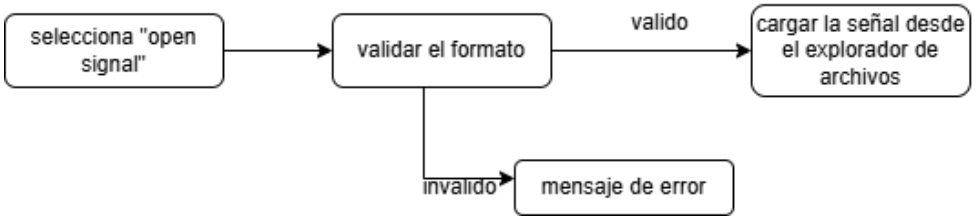


Figura 19: Flujo de carga de la señal (*Open Signal*)

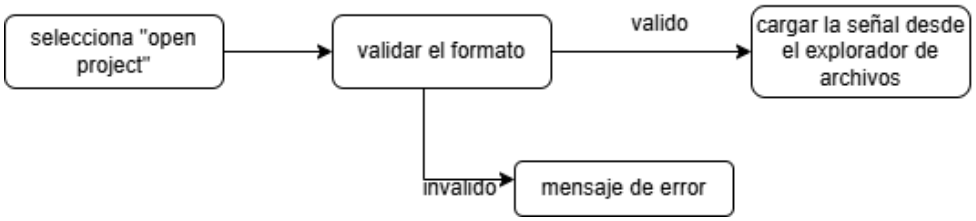


Figura 20: Flujo de carga de proyecto (*Open Project*)

Mediante las opciones *Save* y *Save As* es posible almacenar el estado completo del espacio de trabajo, incluyendo la señal activa, los trials configurados, las medidas realizadas y los resultados de los análisis ejecutados, en un archivo de proyecto con extensión `.glab`.

5.3. Preprocesamiento

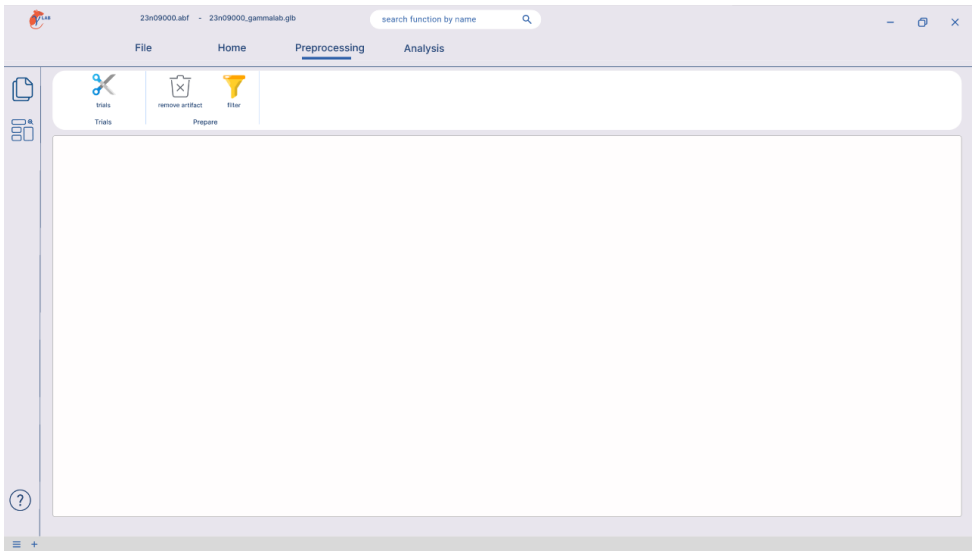


Figura 21: Pestaña Preprocessing de Gamma Lab

Categoria *Trials*

- **trials:** divide la señal cargada en ensayos (*trials*), permitiendo seleccionar el canal a dividir, el canal de estímulos, el umbral (*threshold*), el número de estímulos y el tamaño de la ventana de segmentación. Una vez generados, el usuario puede navegar entre los trials resultantes e incluirlos o descartarlos individualmente del conjunto que participará en los análisis posteriores.

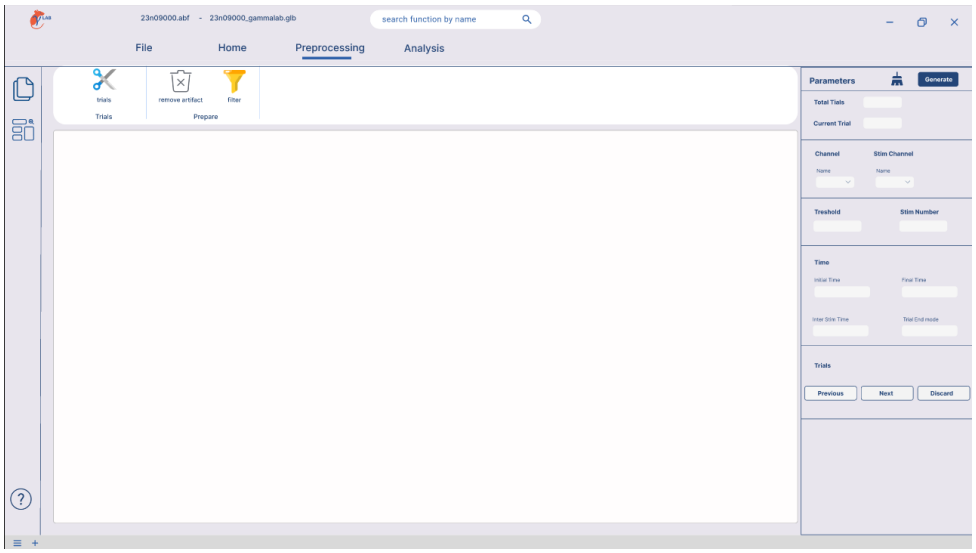


Figura 22: opciones de trials

Categoria *prepare*

- **remove artifact:** elimina artefactos presentes en los trials mediante la selección de un rango de tiempo a corregir, ofreciendo dos modos de corrección: interpolación del segmento seleccionado o eliminación de la señal desde el inicio del trial hasta el punto seleccionado.

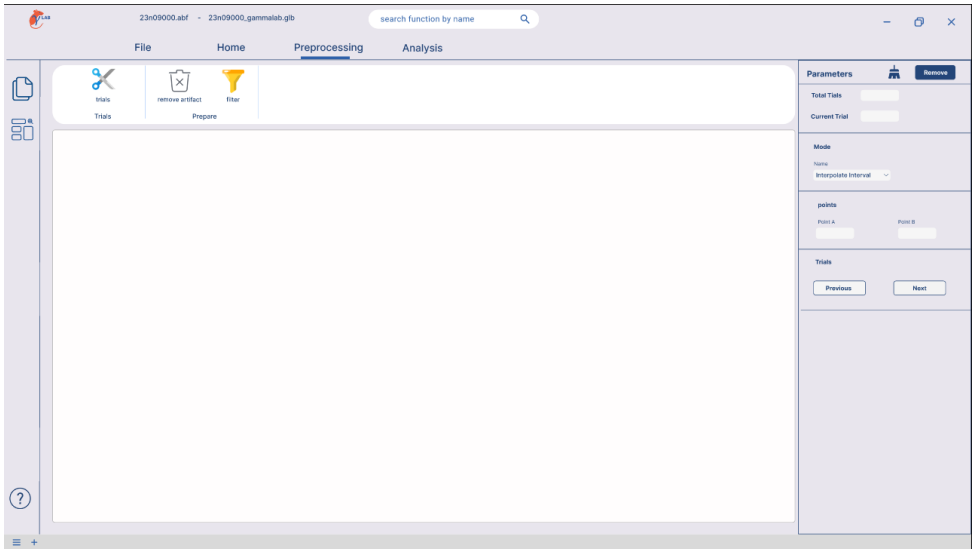


Figura 23: opciones de remove artifact

- **filter:** aplica un filtro digital Butterworth a la señal, mediante la selección del rango de frecuencias de corte y el orden del filtro, mostrando el resultado tanto sobre la señal completa como sobre el primer trial generado.

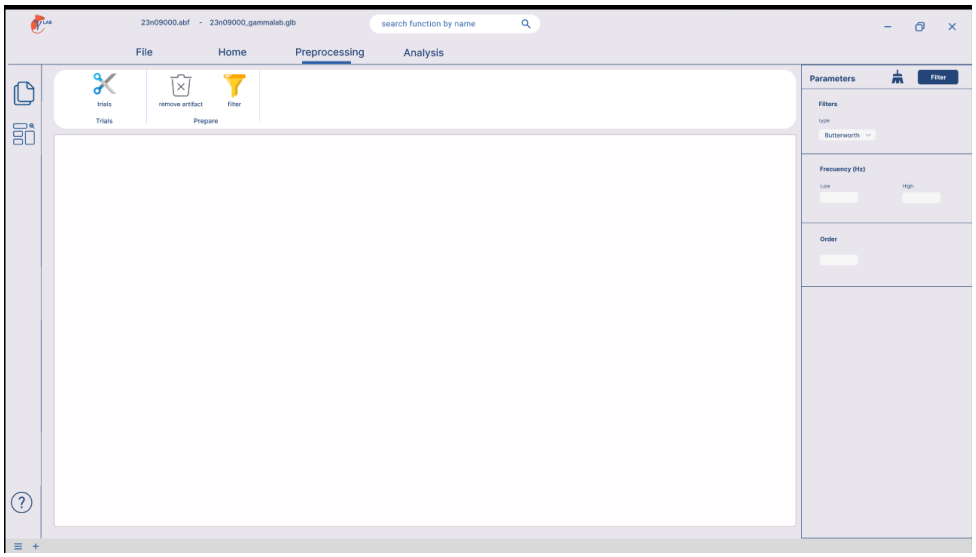


Figura 24: opciones de filter

5.4. Analysis

En esta sección se pueden realizar diferentes análisis de la señal, a continuación seran listados los análisis disponibles y sus opciones:

- **FFT:** calcula la transformada rápida de Fourier de la señal, mediante la configuración de la frecuencia de muestreo y el rango de frecuencias de interés, y muestra el espectro de potencia resultante de forma individual para cada ensayo.

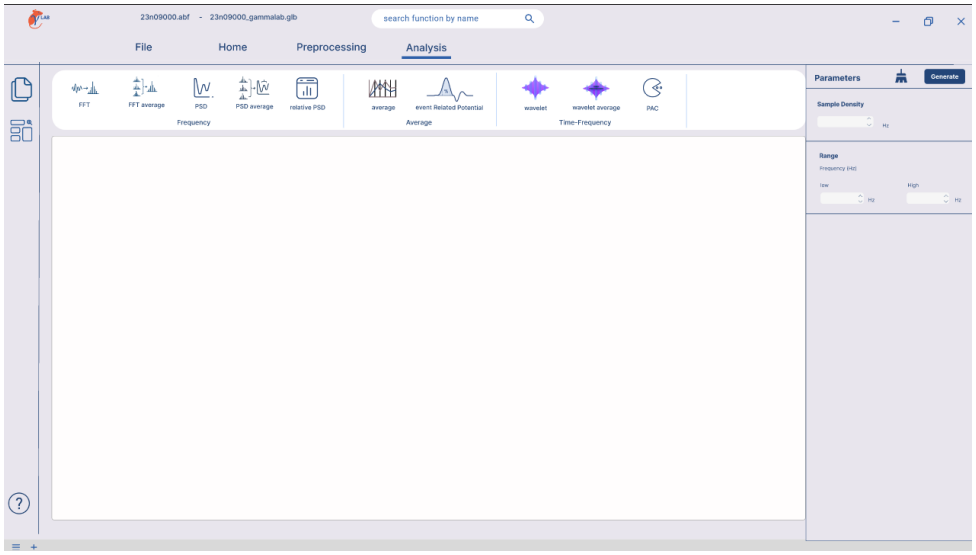


Figura 25: opciones de FFT

- **FFT Average:** calcula la FFT de la señal con la misma configuración de frecuencia de muestreo y rango de frecuencias que la opción anterior, pero adicionalmente promedia los espectros obtenidos de todos los ensayos para reducir el ruido y resaltar los componentes de frecuencia comunes entre ellos.

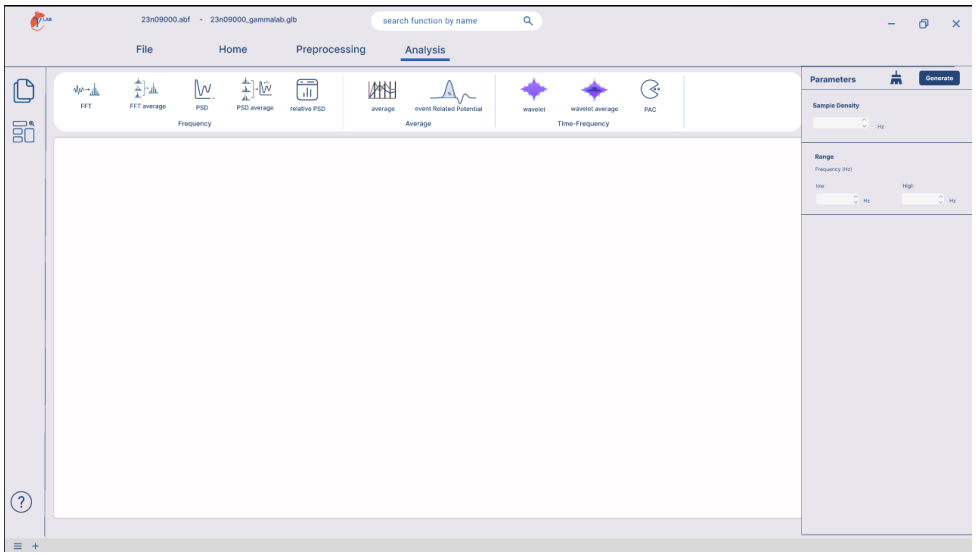


Figura 26: opciones de FFT Average

- **PSD:** estima la densidad espectral de potencia de la señal mediante el método de Welch, permitiendo configurar el modo de cálculo, el tipo de ventana, el rango de frecuencias y el modo de *detrend*, y muestra el resultado de forma individual para cada ensayo.

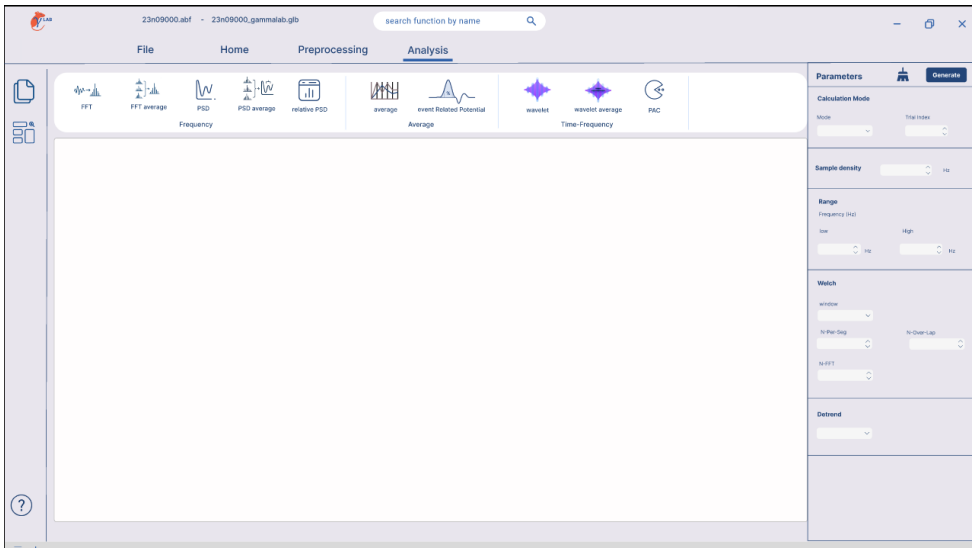


Figura 27: opciones de PSD

- **PSD Average:** estima la PSD de la señal mediante el método de Welch, con la misma configuración de frecuencia de muestreo, ventana y rango de frecuencias que la opción anterior, pero promediando de forma agregada los resultados de todos los ensayos.

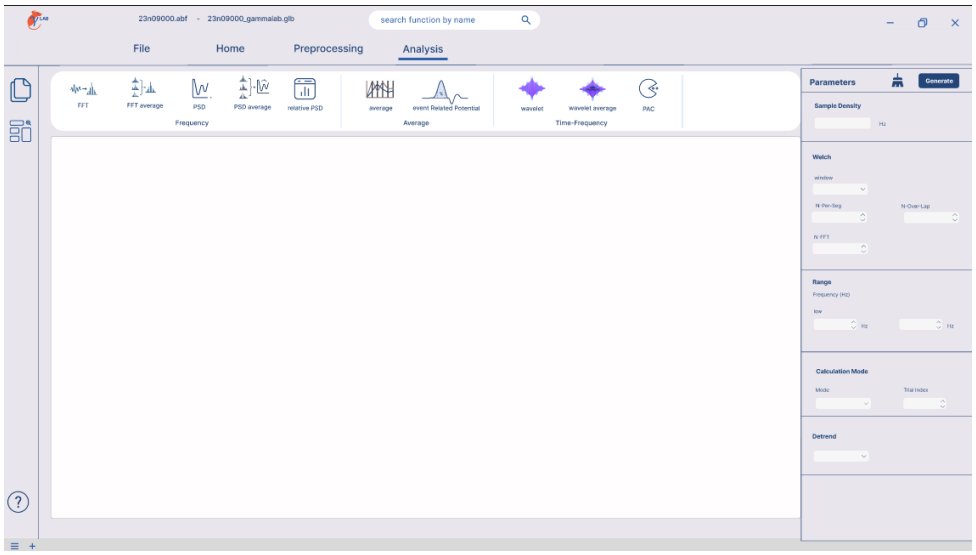


Figura 28: opciones de PSD Average

- **PSD relative:** calcula la PSD relativa de la señal, entregando tanto la potencia absoluta como la potencia relativa aportada por cada banda de frecuencia del rango seleccionado, mediante la configuración del tipo de ventana, el modo de *detrend* y el modo de cálculo.
- **Average:** calcula y visualiza el promedio temporal de los trials generados, útil para observar patrones comunes y atenuar variaciones aleatorias entre ensayos.

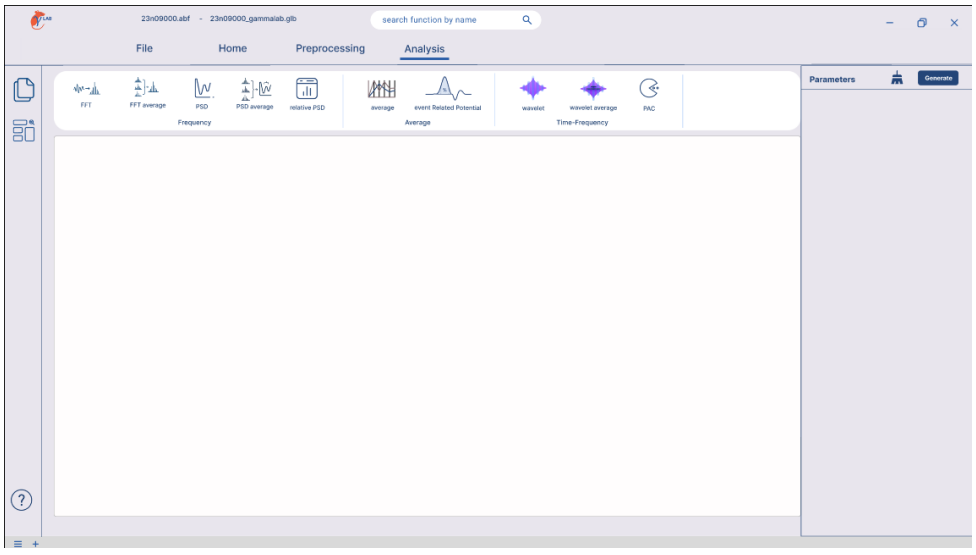


Figura 29: opciones de Average

- **Event related potential:** calcula el potencial relacionado con eventos (ERP), generando un *Butterfly Plot* con todos los trials alineados y un espectrograma de la dinámica tiempo-frecuencia asociada al evento.

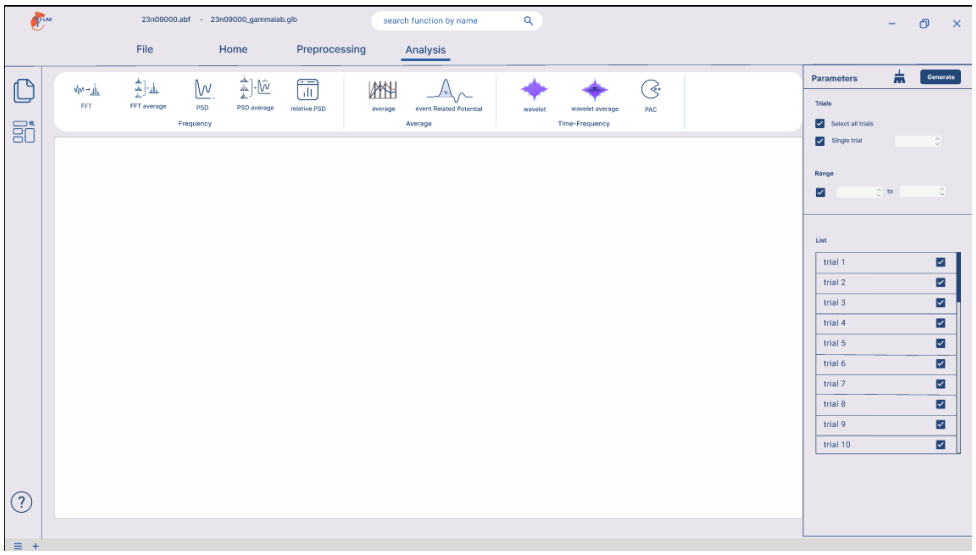


Figura 30: opciones de Event related potential

- **Wavelet:** calcula la transformada wavelet continua de la señal, mediante la configuración de la frecuencia de muestreo, el rango de frecuencias, el número de ciclos, la normalización y la escala de visualización, permitiendo observar cómo varía el contenido espectral de la señal a lo largo del tiempo para cada ensayo.

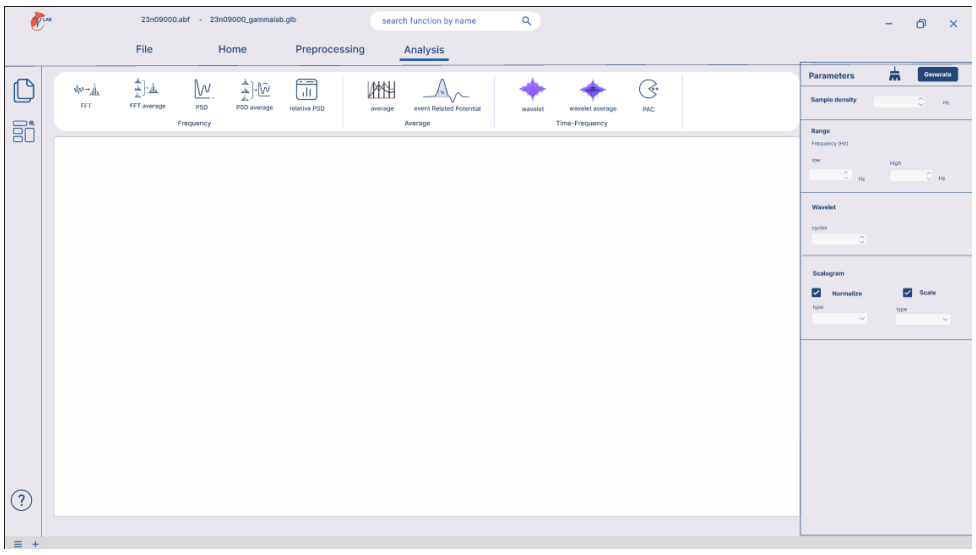


Figura 31: opciones de Wavelet

- **Wavelet Average:** calcula la transformada wavelet con la misma configuración que la opción anterior, y promedia los resultados sobre múltiples trials, generando un mapa tiempo-frecuencia agrupado.

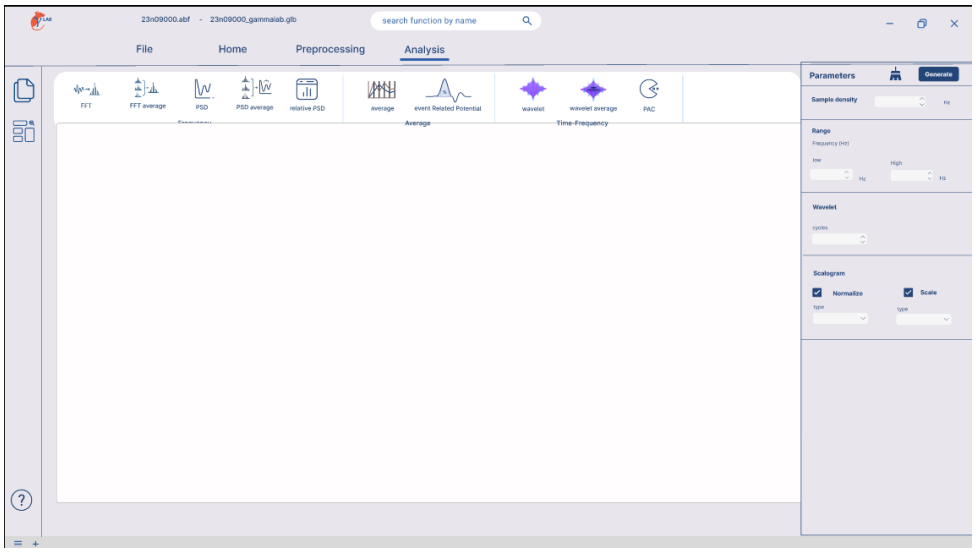


Figura 32: opciones de Wavelet Average

- **PAC:** calcula el acoplamiento de fase-amplitud (*Phase-Amplitude Coupling*) de un canal, evaluando la relación entre la fase de una banda de baja frecuencia y la amplitud de un rango de frecuencias seleccionado, ya sea sobre un trial individual o sobre el promedio de varios trials. El resultado incluye el mapa de acoplamiento fase-frecuencia, el histograma de distribución de amplitud en función de la fase, y el vector de acoplamiento medio con su magnitud y ángulo.

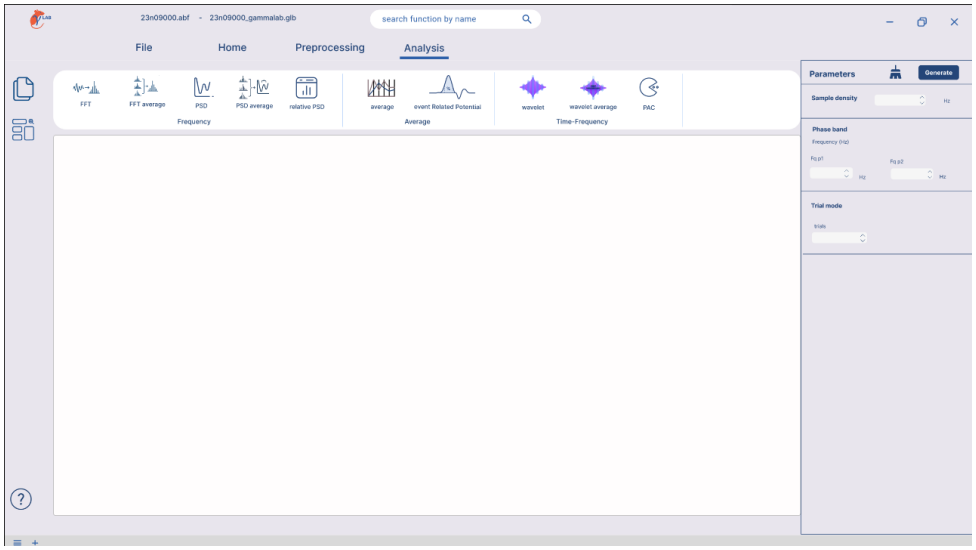


Figura 33: opciones de PAC